

# Rapido Intelligent Mobility Insights

What it does, how to run it, and a full code walkthrough

# Rapido: Intelligent Mobility Insights

---

Ride pattern analysis, cancellation/delay risk prediction, and fare forecasting for a ride-hailing platform, built as an end-to-end pipeline: data cleaning -> feature engineering -> four trained models -> a SQL database -> an EDA report -> an interactive Streamlit dashboard.

## What this project does

---

Given a booking's details (pickup/drop, vehicle type, time, traffic, weather), predicts whether the ride will complete, cancel, or go incomplete; estimates the fare before the trip starts; and scores how likely the customer is to cancel or the driver is to run late - all from four separately-trained models you can query interactively. The dashboard also answers the "what's actually going on" questions (peak hours, which cities cancel more, which vehicle types are most reliable) without writing any code or SQL yourself.

## A note on the dataset

---

This project ships two very different raw datasets, and only one of them is actually usable for the models the spec asks for:

- **data/raw/rides\_data.csv** - ~50,000 real ride records, but with no `customer_id/driver_id`, no traffic/weather/surge columns, and a different fare breakdown (`ride_charge + misc_charge = total_fare`). It's large, but it can't support three of the four required models - there's no way to compute a customer's historical cancellation rate or a driver's delay history without a customer/driver identifier.
- **The bookings/customers/drivers/location\_demand/time\_features tables** (what everything below actually uses) - originally shipped as a 15-booking hand-written sample with exactly the columns every model needs (fare, surge, traffic, weather, driver delay, cancellation reason), but far too small to train anything meaningfully or trust a reported accuracy number on.

`src/generate_synthetic_data.py` regenerates that second dataset at a realistic scale (6,000 bookings, 400 customers, 150 drivers) in the exact same schema, with cancellation/delay probabilities that actually depend on the customer's/driver's history and the trip's traffic/weather - so the models have real signal to learn and the accuracy/RMSE numbers in `reports/MODEL_EVALUATION.md` mean something. `rides_data.csv` is left in `data/raw/` but unused; nothing in the pipeline reads it.

## Objectives

---

- Predict ride outcome (Completed / Cancelled / Incomplete)
- Predict the expected fare before a trip starts
- Score customer cancellation risk and driver delay risk
- Explore ride patterns, cancellations, and fares through a written EDA report and an interactive dashboard
- Store the cleaned data in a real, queryable SQL database

## Folder Structure

---

```
rapido-insights/
|--- data/raw/           raw CSVs (bookings, customers, drivers, location_demand, time_features)
|--- data/processed/     cleaned + merged CSVs (generated by the pipeline)
|--- db/                 schema.sql, analysis_queries.sql, rapido.db (generated)
|--- models/             trained model + encoder .pkl files (generated)
|--- reports/            EDA_REPORT.md, MODEL_EVALUATION.md, charts (generated)
|--- src/                every pipeline script
|--- app/streamlit_app.py the dashboard
```

## Setup

```
pip install -r requirements.txt
```

## Running the Pipeline

Run these from the project root, in order:

```
# 1. (Optional) regenerate the raw CSVs at a larger, realistic scale
python src/generate_synthetic_data.py

# 2. Clean the raw data
python src/data_cleaning.py

# 3. Merge tables and engineer features
python src/feature_engineering.py

# 4. Train all four models + write reports/model_metrics.json and MODEL_EVALUATION.md
python src/train_model.py

# 5. Generate the EDA charts + written report
python src/eda.py

# 6. Load the cleaned data into a local SQLite database
python src/load_db.py

# 7. Launch the dashboard
streamlit run app/streamlit_app.py
```

Step 1 is optional if you already have raw CSVs in `data/raw/` in the same schema - skip straight to step 2. Steps 2-6 can be run independently of each other in any order *after* their own prerequisites are met (EDA only needs step 3's output; the database load only needs step 2's output).

The dashboard opens at <http://localhost:8501>.

## Models

| Model           | Type                                     | File                                       |
|-----------------|------------------------------------------|--------------------------------------------|
| Ride Outcome    | Multiclass classification (RandomForest) | <code>models/ride_outcome_model.pkl</code> |
| Fare Prediction | Regression (XGBoost)                     | <code>models/fare_model.pkl</code>         |

|                            |                                      |                                               |
|----------------------------|--------------------------------------|-----------------------------------------------|
| Customer Cancellation Risk | Binary classification (RandomForest) | <code>models/customer_cancel_model.pkl</code> |
| Driver Delay Risk          | Binary classification (RandomForest) | <code>models/driver_delay_model.pkl</code>    |

Every model is tuned with a small `GridSearchCV` sweep (see `src/train_model.py`) rather than fixed hyperparameters, and every model's test-set metrics are written to `reports/model_metrics.json` and `reports/MODEL_EVALUATION.md` so they're checkable later rather than only ever appearing once in a terminal.

## Benchmark results (target: 85-90% classification accuracy, RMSE within +/-10% of mean fare)

| Model                      | Result                          | vs. benchmark |
|----------------------------|---------------------------------|---------------|
| Ride Outcome               | 65.8% accuracy, AUC 0.60        | Below target  |
| Fare Prediction            | RMSE 5.8% of mean fare, R2=0.99 | Meets target  |
| Customer Cancellation Risk | 82.5% accuracy, AUC 0.61        | Below target  |
| Driver Delay Risk          | 92.6% accuracy, AUC 0.83        | Meets target  |

Fare prediction and driver delay risk clear the target. Ride outcome and customer cancellation risk don't - and honestly, that tracks: both are predicting an inherently noisy human decision (whether *this specific* booking gets cancelled) from aggregate historical signals, and the majority-class baseline (always guessing "Completed" / "No Cancel") already sits at 69%/83% - so there's real, if modest, lift over that baseline, just not enough to hit an 85-90% target on tasks this noisy. Re-run `python src/train_model.py` any time the underlying data changes to refresh these numbers.

## SQL Database

`db/schema.sql` defines a normalized schema - `customers`, `drivers`, `location_demand`, and `time_features` as dimension tables, `bookings` as the fact table referencing all four by foreign key, plus indexes on the columns the dashboard and EDA actually filter/group by (`customer_id`, `driver_id`, `city`, `ride_status`, `booking_time`).

`src/load_db.py` applies that schema to a local SQLite file (`db/rapido.db`) and loads the cleaned CSVs into it - SQLite specifically so running this doesn't require installing or configuring a database server; the DDL itself is plain enough to run on Postgres/MySQL with minor type changes if this ever needs a real server.

`db/analysis_queries.sql` has ready-to-run queries for the EDA-style questions (cancellation rate by city, by traffic level, by weather; payment method usage; top drivers by delay; top customers by cancellation rate). Run them with any SQLite client, or from Python:

```
import sqlite3
conn = sqlite3.connect("db/rapido.db")
for row in conn.execute("SELECT city, COUNT(*) FROM bookings GROUP BY city"):
    print(row)
```

## EDA

`src/eda.py` answers the seven EDA questions from the project spec - ride volume by hour/weekday/city, a cancellation-rate heatmap across cities and time-of-day, distance-vs-fare correlation, rating distributions, a customer-vs-driver cancellation comparison, payment method usage, and traffic/weather vs cancellation rate. It writes `reports/EDA_REPORT.md` with the actual numbers and saves every chart to `reports/eda_charts/`.

`notebooks/EDA.ipynb` predates this script and isn't actually a valid notebook (it's plain Python text saved with a `.ipynb` extension - it won't open in Jupyter). It's safe to delete; `src/eda.py` is the real EDA deliverable.

## Demo Walkthrough

After running the full pipeline, `python src/predict.py` runs all four models against a real row from the processed dataset as a sanity check:

```
Ride outcome: Completed
Fare: 62.36
Customer cancel risk: (0, 0.1033)
Driver delay risk: (0, 0.13)
```

In the dashboard itself: open the **Dashboard** page for headline metrics and a live read of `reports/model_metrics.json`; **EDA** for a quick city-filtered look at the data (the full write-up is in `reports/EDA_REPORT.md`); and the four prediction pages to try a booking against each model interactively - each one starts from a real booking as a template and only asks you to change the handful of fields relevant to that prediction, so you don't have to fill in all ~50 columns by hand.

## Troubleshooting

| Problem                                                             | Solution                                                                                                                                                   |
|---------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>streamlit</code> command not found                            | Use <code>python -m streamlit run app/streamlit_app.py</code>                                                                                              |
| <code>FileNotFoundError</code> for a processed/ CSV or a model .pkl | Run the missing earlier pipeline step (see Running the Pipeline above)                                                                                     |
| <code>ModuleNotFoundError</code>                                    | <code>pip install -r requirements.txt</code>                                                                                                               |
| Dashboard prediction pages error out                                | Make sure <code>train_model.py</code> was re-run after the most recent <code>feature_engineering.py</code> run, so the models match the current column set |

# Rapido: Intelligent Mobility Insights -- How to Run It & Code Walkthrough

---

This document has two parts:

1. **How to run the project**, start to finish, on a clean machine.
2. **A line-by-line explanation of every script** -- `src/utils.py`, `src/generate_synthetic_data.py`, `src/data_cleaning.py`, `src/feature_engineering.py`, `src/train_model.py`, `src/predict.py`, `src/load_db.py`, `src/eda.py`, `app/streamlit_app.py`, and `db/schema.sql` -- so you can follow exactly what each line does and why.

For the "what does the spec ask for vs. what does this project actually have" context (the dataset situation, benchmark results, etc.), see [README.md](#). This document is purely about the code.

---

## Part 1 -- How to Run the Project

---

### 1. Prerequisites

- Python 3.9+
- `data/raw/bookings.csv`, `customers.csv`, `drivers.csv`, `location_demand.csv`, `time_features.csv` already in place (they ship with the repo, or regenerate them with step 3 below)

### 2. Install dependencies

```
pip install -r requirements.txt
```

Installs `pandas`, `numpy`, `scikit-learn`, `matplotlib`, `seaborn`, `streamlit`, `joblib`, `xgboost`, and `openpyxl`.

### 3. Run the pipeline, in this order, from the project root

```
# 1. (Optional) regenerate the raw CSVs at a larger, realistic scale
python src/generate_synthetic_data.py

# 2. Clean the raw data
python src/data_cleaning.py

# 3. Merge tables and engineer features
python src/feature_engineering.py

# 4. Train all four models, tune hyperparameters, write evaluation reports
python src/train_model.py

# 5. Generate the EDA charts and written report
python src/eda.py

# 6. Load the cleaned data into a local SQLite database
python src/load_db.py

# 7. Launch the dashboard
```

```
streamlit run app/streamlit_app.py
```

Step 1 is optional if you're happy with whatever's already in `data/raw/`. Steps 5 and 6 only need step 3's output (`final_merged_data.csv` for EDA, the `_cleaned.csv` files for the database) - they don't depend on step 4, so you can run them in either order relative to training.

Every script is meant to be run with your terminal's working directory at the project root (not inside `src/`) - `python src/data_cleaning.py`, not `cd src && python data_cleaning.py`. `src/` resolves every path from its own file location rather than the current working directory specifically so this doesn't matter, but the commands above assume the project root regardless, since that's also where `streamlit run` needs to be run from to find `data/` and `models/`.

## 4. Using the dashboard

Six pages, picked from the sidebar dropdown: **Dashboard** (headline metrics, ride volume, and a live read of the saved model metrics), **EDA** (quick interactive city filter - the full write-up lives in `reports/EDA_REPORT.md`), and four prediction pages, one per model. Each prediction page only asks you to fill in the handful of fields relevant to that model; everything else is filled in from a real booking behind the scenes (see `build_template_row()` in Part 2).

## 5. If something goes wrong

| Symptom                                                                                              | Fix                                                                                                                                                |
|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>streamlit</code> command not found                                                             | <code>python -m streamlit run app/streamlit_app.py</code>                                                                                          |
| <code>FileNotFoundError</code> for a <code>_cleaned.csv</code> or <code>final_merged_data.csv</code> | Run the missing earlier pipeline step                                                                                                              |
| <code>FileNotFoundError</code> for a model <code>.pkl</code>                                         | Run <code>python src/train_model.py</code>                                                                                                         |
| <code>ModuleNotFoundError</code>                                                                     | <code>pip install -r requirements.txt</code>                                                                                                       |
| Dashboard prediction pages raise a column-mismatch error                                             | Re-run <code>train_model.py</code> after the most recent <code>feature_engineering.py</code> run, so the saved models match the current column set |

## Part 2 -- Code Walkthrough

### `src/``utils.py`

Shared path constants and small file-I/O helpers every other script imports from. Nothing here is specific to any one pipeline stage - it exists so path handling and file-not-found errors are written once rather than copy-pasted into six files.

### Lines 1-13 -- module docstring, imports, path constants

```
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
RAW_DIR = os.path.join(BASE_DIR, "data", "raw")
PROCESSED_DIR = os.path.join(BASE_DIR, "data", "processed")
MODELS_DIR = os.path.join(BASE_DIR, "models")
```

```
REPORTS_DIR = os.path.join(BASE_DIR, "reports")
DB_DIR = os.path.join(BASE_DIR, "db")
```

`os.path.abspath(__file__)` resolves to the absolute path of `utils.py` itself, no matter what directory the terminal happened to be in when the script that imported it was launched. `os.path.dirname(...)` peels off one path component at a time: the first call strips `utils.py` down to the `src/` folder, and wrapping it a second time strips `src/` down to the project root. Every other directory constant is then built from that project-root `BASE_DIR` with `os.path.join` (which correctly uses `\` on Windows and `/` elsewhere, rather than hard-coding either). This is exactly what fixes the bug mentioned in the README - earlier versions of `feature_engineering.py/train_model.py` passed the literal string `"data/processed"` instead, which only resolves correctly if the terminal's current directory happens to be the project root; using `PROCESSED_DIR` here makes every path absolute and therefore correct regardless of where you run the script from.

#### Lines 16-20 -- `ensure_dirs()`

```
def ensure_dirs():
    for path in (PROCESSED_DIR, MODELS_DIR, REPORTS_DIR, DB_DIR):
        os.makedirs(path, exist_ok=True)
```

Creates every output folder the pipeline writes to. `exist_ok=True` means calling this a second (or tenth) time across separate script runs doesn't raise `FileExistsError` - it's a no-op once the folders are already there. `RAW_DIR` is deliberately not in this list - the raw input folder is expected to already exist with real data in it; there's nothing useful about auto-creating an empty one.

#### Lines 23-35 -- `load_csv()`

```
def load_csv(filename, folder=RAW_DIR):
    path = os.path.join(folder, filename)
    try:
        return pd.read_csv(path)
    except FileNotFoundError as exc:
        raise FileNotFoundError(
            f"Could not find {path}. If this is a processed/ file, run the "
            f"earlier pipeline steps first (data_cleaning.py, "
            f"feature_engineering.py)."
        ) from exc
```

`folder=RAW_DIR` as a default means every call site that doesn't specify a folder is implicitly reading from `data/raw/` - which is why `data_cleaning.py`'s calls to `load_csv(filename)` need no folder argument at all, while `feature_engineering.py` and `train_model.py` explicitly pass `folder=PROCESSED_DIR` to read from the processed-data folder instead. The `try/except` re-raises the same exception type (`FileNotFoundError`) with a more specific message, rather than swallowing it or raising something generic - callers that already catch `FileNotFoundError` (like `streamlit_app.py`) keep working unchanged, they just get a clearer message when it fires. `raise ... from exc` preserves the original traceback as the new exception's cause, so you can still see exactly which underlying `pd.read_csv` call failed if you need to debug further.

#### Lines 38-44 -- `save_csv()`

```
def save_csv(df, filename, folder=PROCESSED_DIR):
    os.makedirs(folder, exist_ok=True)
    path = os.path.join(folder, filename)
    df.to_csv(path, index=False)
    print(f"Saved file: {path}")
    return path
```



The opposite default from `load_csv - folder=PROCESSED_DIR`, since every script that saves a CSV is saving a *processed* output, not raw input. `os.makedirs(folder, exist_ok=True)` runs again here (in addition to `ensure_dirs()`) as a defensive belt-and-suspenders measure - if this function is ever called from a script that forgot to call `ensure_dirs()` first, saving still won't fail with a missing-directory error. `index=False` skips writing pandas' auto-generated row index (0, 1, 2, ...) as a spurious extra column in the CSV. Returning `path` lets a caller log or inspect exactly where the file landed, though nothing in this codebase currently uses that return value.

#### Lines 47-51 -- `save_raw_csv()`

```
def save_raw_csv(df, filename):  
    return save_csv(df, filename, folder=RAW_DIR)
```

A one-line wrapper that redirects `save_csv` to write into `data/raw/` instead of `data/processed/`. Only `generate_synthetic_data.py` calls this - it's the one script in the whole pipeline that *produces* raw input data rather than consuming and transforming it, so it needs the one save function pointed the opposite direction from every other script's writes.

---

#### `src/generate_synthetic_data.py`

Builds a realistic-scale version of the five raw tables from scratch. This exists because the dataset that shipped with the project had exactly the right *columns* for every model but only 15 booking rows - nowhere near enough to train anything or trust a reported metric on (see README for the full reasoning, including why the separate 50k-row `rides_data.csv` couldn't be used instead).

#### Lines 21-26 -- imports

```
import os  
import numpy as np  
import pandas as pd  
from utils import ensure_dirs, save_raw_csv
```

`os` is imported but this file never actually calls anything from it directly - `ensure_dirs()` (from `utils.py`) does the directory creation instead. `numpy (np)` drives every random draw and clip; `pandas` builds the final DataFrames.

#### Lines 28-54 -- configuration constants

```
RNG_SEED = 42  
N_CUSTOMERS = 400  
N_DRIVERS = 150  
N_BOOKINGS = 6000  
  
CITIES = ["Delhi", "Bengaluru", "Mumbai", "Hyderabad", "Pune"]  
LOCATIONS_BY_CITY = { ... }  
VEHICLE_TYPES = ["Bike", "Auto", "Cab"]  
PAYMENT_METHODS = ["UPI", "Cash", "Card", "Wallet"]  
TRAFFIC_LEVELS = ["Low", "Medium", "High"]  
WEATHER_CONDITIONS = ["Clear", "Cloudy", "Rain", "Smog"]  
ZONE_TYPES = ["Residential", "Commercial", "Tourist", "Transit"]  
BASE_FARE_BY_VEHICLE = {"Bike": 40, "Auto": 60, "Cab": 90}  
PER_KM_BY_VEHICLE = {"Bike": 6, "Auto": 9, "Cab": 14}
```

`RNG_SEED = 42` matters more than it looks - every random draw in this file goes through a single `np.random.default_rng(RNG_SEED)` instance created once in `generate_all()` and threaded through every helper

function as the `rng` argument, so running this script twice produces *identical* output both times. That reproducibility is what makes the model metrics in `reports/MODEL_EVALUATION.md` trustworthy from one run to the next, rather than shifting every time someone regenerates the data. `LOCATIONS_BY_CITY` gives each city its own pool of realistic neighborhood names rather than one global list, so a Delhi booking's pickup/drop pair is always a real Delhi locality pair, never an accidental Delhi-to-Bengaluru mismatch. `BASE_FARE_BY_VEHICLE/PER_KM_BY_VEHICLE` give each vehicle type a different pricing structure (a Cab both costs more to start and more per km than a Bike), which is what makes the "vehicle-type bands" visible in the distance-vs-fare EDA chart.

#### Lines 57-75 -- `_make_time_features()`

```
def _make_time_features():
    rows = []
    for hour in range(24):
        is_peak = int(hour in (7, 8, 9, 17, 18, 19))
        if 5 <= hour < 11:
            bucket = "Morning"
        elif 11 <= hour < 14:
            bucket = "Late Morning"
        elif 14 <= hour < 17:
            bucket = "Afternoon"
        elif 17 <= hour < 21:
            bucket = "Evening"
        else:
            bucket = "Night"
        rows.append({"hour": hour, "is_peak_hour": is_peak, "is_weekend": 0, "time_bucket": bucket})
    return pd.DataFrame(rows)
```

Builds a fixed 24-row lookup table, one row per hour of the day - this is a dimension table, not something that needs to scale with the number of bookings, so it's the one table in this file that isn't parameterized by `N_BOOKINGS` or similar. The if/elif chain buckets each hour into one of five human-readable labels; `is_peak_hour` flags the specific hours (7-9am, 5-7pm) that later feed into higher traffic/surge probabilities in `_make_bookings()`. `is_weekend` is hard-coded to `0` here because weekend-ness is a property of a specific *date*, not of an hour-of-day in the abstract - the real per-booking weekend flag gets computed later in `feature_engineering.py` from each booking's actual timestamp, and this column exists here only so the table has the same shape the original project's `time_features.csv` did.

#### Lines 78-99 -- `_make_customers()`

```
def _make_customers(rng):
    rows = []
    for i in range(1, N_CUSTOMERS + 1):
        completed = int(rng.integers(2, 300))
        cancel_rate = np.clip(rng.beta(2, 14), 0.0, 0.6)
        cancelled = int(round(completed * cancel_rate / max(1 - cancel_rate, 0.01)))
        rating = np.clip(rng.normal(4.3 - cancel_rate, 0.3), 1.0, 5.0)
        rows.append({...})
    return pd.DataFrame(rows)
```

Generates one row per customer with a set of *internally consistent* attributes rather than independently random ones. `rng.integers(2, 300)` picks how many rides this customer has completed historically. `rng.beta(2, 14)` is the important line - each customer gets a persistent "flakiness" trait drawn from a Beta distribution, which is right-skewed: most draws cluster low (a typical customer rarely cancels) but the tail extends out to higher values (a meaningful minority of customers cancel often). This replaced an earlier version that used a tight `rng.normal(...)` with a small spread, which produced customers whose cancellation rates were all so close together that the eventual `customer_cancellation_rate` feature barely differed between rides that got cancelled and rides that didn't - giving the cancellation-risk model in `train_model.py` almost nothing to learn from. `cancelled = round(completed * cancel_rate / (1 - cancel_rate))` backs out how many cancelled rides would produce that target rate given the

completed count (algebra: if `cancel_rate = cancelled / (completed + cancelled)`, solving for `cancelled` gives this formula). `rating = 4.3 - cancel_rate` links a customer's star rating to the same underlying trait - flakier customers tend to have lower ratings, mirroring how a real platform's data would plausibly correlate the two, so the `customer_rating` column also carries some of the same predictive signal for a model that never sees the underlying "flakiness" number itself.

#### Lines 102-122 -- `_make_drivers()`

```
def _make_drivers(rng):
    rows = []
    for i in range(1, N_DRIVERS + 1):
        completed = int(rng.integers(20, 800))
        acceptance = np.clip(rng.normal(0.90, 0.08), 0.4, 1.0)
        avg_delay = np.clip(rng.normal(8 - acceptance * 6, 3), 0, 30)
        cancelled = int(rng.integers(0, max(int(completed * 0.15), 1)))
        rating = np.clip(rng.normal(4.5 - avg_delay / 25, 0.25), 1.0, 5.0)
        rows.append({...})
    return pd.DataFrame(rows)
```

Same "one persistent trait drives several correlated columns" pattern as customers, but built around acceptance rate instead of a cancellation trait. `avg_delay = 8 - acceptance * 6` means a driver who accepts nearly every ride offered (`acceptance` near 1.0) tends toward a low average delay ( $8 - 6 = 2$  minutes), while a driver who declines a lot of rides (`acceptance` near 0.4) tends toward a higher one ( $8 - 2.4 = 5.6$ , before the added normal noise) - the comment at line 107-108 spells out the intent directly: this is what gives `driver_reliability_score` (computed later in `feature_engineering.py` from `driver_rating`, `acceptance_rate`, and `avg_delay_min` together) actual predictive power for the driver-delay model, rather than being three independently-random numbers with nothing tying them together.

#### Lines 125-143 -- `_make_location_demand()`

```
def _make_location_demand(rng):
    rows = []
    seen_pairs = set()
    for city, spots in LOCATIONS_BY_CITY.items():
        for pickup in spots:
            for drop in spots:
                if pickup == drop or (pickup, drop) in seen_pairs:
                    continue
                seen_pairs.add((pickup, drop))
                ...
    return pd.DataFrame(rows)
```

A nested loop over every pickup/drop combination *within* the same city (the outer loop is over cities, and both inner loops draw from that same city's `spots` list - locations from different cities never pair up, since nobody books a ride from a Delhi neighborhood to a Bengaluru one in this dataset). `if pickup == drop: continue` skips the nonsensical case of a trip that starts and ends at the same spot. `seen_pairs` is a `set` of `(pickup, drop)` tuples used purely to guard against generating the exact same pair twice - redundant here since nested loops over the same list naturally can't repeat a pair, but a defensive habit worth keeping if the location lists ever became overlapping or generated dynamically.

#### Lines 146-249 -- `_make_bookings()`

The core generator - one row per booking, and by far the most involved function in the file since it has to produce every column every downstream model reads.

```
customer_weights = (customers["total_completed_rides"] + 1).to_numpy()
customer_weights = customer_weights / customer_weights.sum()
driver_weights = (drivers["total_completed_rides"] + 1).to_numpy()
driver_weights = driver_weights / driver_weights.sum()
```

Lines 152-155: rather than assigning each booking to a uniformly random customer/driver, bookings are weighted toward customers/drivers who already have a higher `total_completed_rides` count. The `+ 1` avoids a zero-weight customer ever having literally no chance of being picked. Dividing by the sum turns the raw weights into a probability distribution that sums to 1, which is what `rng.choice(..., p=weights)` requires. The comment explains why this matters: without it, a customer with only 2 historical completed rides could still end up the subject of, say, 40 of this run's 6,000 bookings purely by chance, which would make their supposedly-fixed historical cancellation rate wildly disconnected from their actual behavior in this dataset.

```
for i in range(1, N_BOOKINGS + 1):
    city = rng.choice(CITIES)
    spots = LOCATIONS_BY_CITY[city]
    pickup, drop = rng.choice(spots, size=2, replace=False)
```

Lines 160-163: for each of the 6,000 bookings, pick a city uniformly at random, then pick two *different* locations from that city's list - `replace=False` is what guarantees pickup and drop are never the same spot (`rng.choice` with `replace=True`, the default, could otherwise return the same location for both draws).

```
day_offset = int(rng.integers(0, 90))
hour = int(rng.integers(0, 24))
minute = int(rng.integers(0, 60))
booking_time = start + pd.Timedelta(days=day_offset, hours=hour, minutes=minute)
```

Lines 165-168: builds a timestamp somewhere in a 90-day window starting 2026-01-01, at a uniformly random hour and minute. This is why the "ride volume by hour" EDA chart comes out nearly flat - hour is drawn uniformly here, so there's no built-in demand spike at rush hour, only a *cancellation/traffic* effect at those hours (via `is_peak` below). That's a deliberate scope choice, not a bug: modeling realistic demand curves wasn't the point of this generator, learnable cancellation/delay signal was.

```
vehicle_type = rng.choice(VEHICLE_TYPES, p=[0.35, 0.35, 0.30])
distance_km = round(float(np.clip(rng.exponential(6) + 1, 0.5, 45)), 2)
is_peak = hour in (7, 8, 9, 17, 18, 19)
traffic_level = rng.choice(
    TRAFFIC_LEVELS, p=[0.2, 0.35, 0.45] if is_peak else [0.5, 0.35, 0.15]
)
```

Lines 170-175: `rng.exponential(6) + 1` draws trip distance from an exponential distribution (mean 6) shifted up by 1 - this produces mostly short trips with a long tail of occasional long ones, which is a much more realistic distance distribution than a uniform or normal draw would give (real ride-hailing trips genuinely look like this). `traffic_level`'s probability list is conditional on `is_peak` - during peak hours, "High" traffic is the most likely outcome (45%); off-peak, "Low" is (50%). This is the mechanism that gives `traffic_level` real predictive power over cancellation later, since it isn't independent of the hour.

```
traffic_speed_factor = {"Low": 1.0, "Medium": 1.3, "High": 1.7}[traffic_level]
trip_duration_min = round(distance_km * traffic_speed_factor * rng.uniform(1.6, 2.4), 1)
```

Lines 178-179: trip duration is derived from distance, scaled up by a traffic-dependent multiplier (heavier traffic means the same distance takes longer) and a random factor standing in for everything else that affects speed (turns, stops, driving style). This is what makes `trip_duration_min` correlate sensibly with both `distance_km` and `traffic_level` instead of being an independent random number.

```

base_fare = round(
    BASE_FARE_BY_VEHICLE[vehicle_type] + distance_km * PER_KM_BY_VEHICLE[vehicle_type], 2
)
surge_multiplier = round(
    1.0
    + (0.4 if is_peak else 0.0)
    + (0.3 if traffic_level == "High" else 0.0)
    + (0.2 if weather_condition == "Rain" else 0.0)
    + rng.uniform(0, 0.15),
    2,
)

```

Lines 181-191: `base_fare` is a flat per-vehicle-type charge plus a per-km rate - a standard taxi-meter style formula. `surge_multiplier` starts at 1.0 (no surge) and stacks additive bumps for peak hours, heavy traffic, and rain, plus a small amount of random noise - so `surge` isn't an independent random number either, it's driven by the same conditions that also drive `traffic_level/is_peak/weather_condition`, which is what gives the fare-prediction model real structure to fit (and is a big part of why that model's R2 comes out at 0.99 - the underlying formula really is close to additive and learnable).

```

customer_id = rng.choice(customer_ids, p=customer_weights)
driver_id = rng.choice(driver_ids, p=driver_weights)
customer_row = customers.loc[customers["customer_id"] == customer_id].iloc[0]
driver_row = drivers.loc[drivers["driver_id"] == driver_id].iloc[0]

```

Lines 193-196: picks this booking's customer and driver using the weighted distributions built earlier, then looks up that customer's/driver's full row from the tables built by `_make_customers()/_make_drivers()` - this is how a booking "knows" its customer's historical cancellation rate and its driver's average delay, needed for the next block.

```

customer_hist_cancel_rate = customer_row["total_cancelled_rides"] / max(
    customer_row["total_completed_rides"] + customer_row["total_cancelled_rides"], 1
)
driver_unreliability = driver_row["avg_delay_min"] / 30 + (1 - driver_row["acceptance_rate"])

cancel_prob = np.clip(
    0.03 + customer_hist_cancel_rate * 1.3 + (0.10 if traffic_level == "High" else 0)
    + (0.10 if weather_condition in ("Rain", "Smog") else 0),
    0.01, 0.75,
)
incomplete_prob = np.clip(0.02 + driver_unreliability * 0.22, 0.01, 0.3)

```

Lines 201-211: this is the heart of the whole generator - the formulas that actually determine whether a booking ends up cancelled or incomplete, and they're built entirely from signals that are also present as columns in the final dataset (the customer's historical cancellation rate, the driver's delay/acceptance history, current traffic, current weather). The specific coefficients (`1.3`, `0.10`, `0.22`) were tuned up from smaller starting values after an earlier version of this file produced models that scored *below random* (AUC under 0.5) - with the original coefficients (`0.5`, `0.05`, `0.05` for cancellation; `0.05` for delay) the systematic signal was so small relative to the random noise in `roll` below that a RandomForest genuinely couldn't separate cancelled bookings from completed ones. There's nothing special about these particular numbers beyond "large enough that the resulting models have real, checkable signal to learn" (see `reports/MODEL_EVALUATION.md` for what that signal turned into in practice) - `np.clip(..., low, high)` just keeps every probability inside a sane range regardless of how extreme the inputs get.

```

roll = rng.random()
if roll < cancel_prob:
    ride_status = "Cancelled"
    cancelled_by = rng.choice(["Customer", "Driver"], p=[0.7, 0.3])
elif roll < cancel_prob + incomplete_prob:
    ride_status = "Incomplete"

```

```

    cancelled_by = "None"
else:
    ride_status = "Completed"
    cancelled_by = "None"

```

Lines 213-222: a single uniform random draw (`roll`, between 0 and 1) decides the outcome by comparing against the two computed probabilities stacked end to end - this is the standard way to sample a 3-outcome categorical result from two probability values without needing a separate random draw per outcome. If `roll` lands below `cancel_prob`, it's cancelled; if it's above that but below `cancel_prob + incomplete_prob`, it's incomplete; otherwise it's completed. Cancelled bookings additionally get a `cancelled_by` label, weighted 70/30 toward the customer - reflecting that customer-initiated cancellations are typically more common than driver-initiated ones on a real platform.

```

driver_delay_min = max(0, round(rng.normal(driver_row["avg_delay_min"], 3), 1))
if traffic_level == "High":
    driver_delay_min += rng.uniform(0, 4)

```

Lines 224-226: this booking's actual delay is drawn from a normal distribution centered on *this driver's* average historical delay (not a global average), with an extra random bump added specifically under high traffic. `max(0, ...)` prevents a negative delay, which wouldn't make physical sense. This is what feeds `driver_delay_flag` (computed later in `feature_engineering.py` as `driver_delay_min >= 10`) real signal tied to both the driver's identity and the trip's traffic conditions.

### Lines 252-273 -- `generate_all()`

```

def generate_all():
    rng = np.random.default_rng(RNG_SEED)
    ensure_dirs()

    customers = _make_customers(rng)
    drivers = _make_drivers(rng)
    location_demand = _make_location_demand(rng)
    time_features = _make_time_features()
    bookings = _make_bookings(rng, customers, drivers, location_demand)

    save_raw_csv(customers, "customers.csv")
    save_raw_csv(drivers, "drivers.csv")
    save_raw_csv(location_demand, "location_demand.csv")
    save_raw_csv(time_features, "time_features.csv")
    save_raw_csv(bookings, "bookings.csv")
    ...

```

The orchestrator - creates one shared random generator (`rng`) and threads it through every `_make_*` helper, which is what makes the *entire* run reproducible from a single seed, not just individual tables in isolation. Customers, drivers, and location demand are built before bookings specifically because `_make_bookings()` needs to read real customer/driver rows to compute `customer_hist_cancel_rate/driver_unreliability` - the generation order here mirrors the foreign-key dependency order later enforced by `db/schema.sql`. `location_demand` is passed into `_make_bookings()` as a parameter but never actually used inside it (a loose end from an earlier design where pickup/drop pairs were meant to be drawn from that table directly, rather than from `LOCATIONS_BY_CITY` inside `_make_bookings()` itself) - harmless, but worth knowing the parameter is currently unused if you're extending this function later.

Cleans each of the five raw CSVs independently and writes a `_cleaned.csv` version of each to `data/processed/`. Every function here takes a DataFrame and returns a DataFrame, chained together in `clean_file()`.

#### Lines 15-21 -- `RAW_FILES`

```
RAW_FILES = [  
    "bookings.csv", "customers.csv", "drivers.csv",  
    "location_demand.csv", "time_features.csv",  
]
```

The five filenames `main()` loops over - named once here rather than inlined into the loop, so adding a sixth raw file later means editing one list instead of hunting through the function body.

#### Lines 24-34 -- `clean_columns()`

```
def clean_columns(df):  
    df.columns = (  
        df.columns.str.strip()  
        .str.lower()  
        .str.replace(" ", "_")  
        .str.replace(r"[^a-z0-9_]", "", regex=True)  
    )  
    return df
```

Chains four string operations across the whole column-name `Index` at once: strip whitespace, lowercase, replace literal spaces with underscores, then strip out anything that isn't a lowercase letter, digit, or underscore. That last regex is the belt-and-suspenders step - after the first three operations, column names should already be clean, but this catches anything else that slipped through (a stray BOM character from an Excel export, punctuation, etc.) rather than trusting the input is perfectly formed.

#### Lines 37-48 -- `fill_missing()`

```
def fill_missing(df):  
    for col in df.columns:  
        if pd.api.types.is_numeric_dtype(df[col]):  
            df[col] = df[col].fillna(df[col].median())  
        else:  
            df[col] = df[col].fillna("Unknown")  
    return df
```

`pd.api.types.is_numeric_dtype(df[col])` rather than checking `df[col].dtype == "object"` - this matters specifically because pandas 3.x stores plain text columns as a `str` dtype, not the older `object` dtype, so a direct `== "object"` comparison silently misses every text column and sends it down the *numeric* branch instead, which then crashes trying to compute a median of strings. Using the type-checking function instead of a literal dtype comparison works correctly across pandas versions. Numeric columns get their own column's median (robust to outliers, unlike a mean); text columns get the literal string `"Unknown"` rather than staying `NaN`, so a missing category doesn't silently vanish from a `groupby` or get dropped by a one-hot encoder later.

#### Lines 51-63 -- `parse_datetime_columns()`

```
def parse_datetime_columns(df):  
    for col in df.columns:  
        if "time" in col or "date" in col:  
            try:  
                df[col] = pd.to_datetime(df[col])
```



```

        except (ValueError, TypeError):
            pass
    return df

```

Any column whose *name* contains "time" or "date" gets a shot at `pd.to_datetime`. The try/except matters because this is a name-based heuristic, not a guarantee - a column like `trip_duration_min` contains "time"-adjacent characters... actually no, it contains neither "time" nor "date" literally, but a hypothetical future column name could coincidentally match without holding real date data, and this guards against that case crashing the whole cleaning run. Catching the *specific* exceptions `ValueError/TypeError` (rather than a bare `except:`) means a genuine bug elsewhere in the parsing logic would still surface as a real crash, instead of being silently swallowed along with the expected "this isn't actually a date" case.

#### Lines 66-74 -- `standardize_status()`

```

def standardize_status(df):
    if "ride_status" in df.columns:
        df["ride_status"] = df["ride_status"].astype(str).str.strip().str.title()
    if "cancelled_by" in df.columns:
        df["cancelled_by"] = df["cancelled_by"].astype(str).str.strip().str.title()
    return df

```

`.str.title()` capitalizes the first letter of each word - "`completed`", "`Completed`", and "`COMPLETED`" all become "`Completed`". Both checks are guarded by `if col in df.columns` since not every one of the five raw files has these columns (only `bookings.csv` does) - this function runs against all five files inside `clean_file()`, so it needs to be a no-op on the four that don't have these columns.

#### Lines 77-84 -- `basic_numeric_clean()`

```

def basic_numeric_clean(df):
    num_cols = df.select_dtypes(include=[np.number]).columns
    for col in num_cols:
        df[col] = df[col].replace([np.inf, -np.inf], np.nan)
    return df

```

Runs *before* `fill_missing()` in the pipeline specifically because `fill_missing()`'s median-fill only catches `NaN`, not literal infinity - a stray divide-by-zero somewhere upstream could otherwise leave an `inf` sitting in a numeric column, silently skewing any median or mean computed from it. Converting `inf/-inf` to `NaN` first means the later median-fill step catches those values too.

#### Lines 87-105 -- `clean_file()`

```

def clean_file(filename):
    df = load_csv(filename)
    before_rows = len(df)

    df = clean_columns(df)
    df = df.drop_duplicates()
    df = parse_datetime_columns(df)
    df = basic_numeric_clean(df)
    df = fill_missing(df)
    df = standardize_status(df)

    after_rows = len(df)
    print(f"{filename}: {before_rows} -> {after_rows} rows")

    cleaned_name = filename.replace(".csv", "_cleaned.csv")

```



```
save_csv(df, cleaned_name)
return df
```

Threads one raw file through every cleaning step in a deliberate order: column names get normalized first (so every later step can refer to predictable snake\_case names), then exact duplicate rows are dropped, then datetime parsing, then infinity-cleanup, then missing-value fill, then status standardization last (since it only touches two specific columns and doesn't depend on anything before it). The before/after row count print exists purely so running this script tells you whether `drop_duplicates()` actually removed anything.

#### Lines 108-118 -- `main()`

```
def main():
    ensure_dirs()
    for filename in RAW_FILES:
        try:
            clean_file(filename)
        except FileNotFoundError as exc:
            print(f"Skipping {filename}: {exc}")
```

Loops over all five raw files, but wraps each one in its own try/except - if one file is missing (say, you deleted `location_demand.csv` by accident), the other four still get cleaned instead of the whole run aborting on the first missing file.

---

#### `src/feature_engineering.py`

Merges the five cleaned tables into one row-per-booking dataset and derives every feature the four models train on. Structured as one function per logical step, chained together in `build_features()`.

#### Lines 18-26 -- `load_cleaned_tables()`

```
def load_cleaned_tables():
    return {
        "bookings": load_csv("bookings_cleaned.csv", folder=PROCESSED_DIR),
        "customers": load_csv("customers_cleaned.csv", folder=PROCESSED_DIR),
        ...
    }
```

Returns a dict keyed by a short name rather than five separate local variables, so `merge_tables()` below can refer to `tables["customers"]` etc. - slightly more verbose at the call site but makes it obvious which table is which without needing to track five positional arguments.

#### Lines 29-36 -- `add_time_parts()`

```
def add_time_parts(bookings):
    bookings["booking_time"] = pd.to_datetime(bookings["booking_time"], errors="coerce")
    bookings["hour"] = bookings["booking_time"].dt.hour
    bookings["day_of_week"] = bookings["booking_time"].dt.day_name()
    bookings["is_weekend"] = bookings["day_of_week"].isin(["Saturday", "Sunday"]).astype(int)
    return bookings
```

`errors="coerce"` turns any unparseable timestamp into `NaT` (pandas' null timestamp) instead of raising - defensive in case a hand-edited CSV ever has a malformed date. `.dt.hour` and `.dt.day_name()` are pandas' datetime accessor properties, pulling the hour-of-day (0-23) and the full weekday name ("`Monday`", etc.) straight out of the

timestamp. `.isin(...)` returns a boolean Series, and `.astype(int)` converts `True/False` into `1/0` - matching the numeric convention every other flag column in this project uses (`is_peak_hour`, `long_distance_flag`, etc. are all 0/1 ints, not booleans).

#### Lines 39-62 -- `merge_tables()`

```
def merge_tables(tables):
    bookings = add_time_parts(tables["bookings"])

    df = bookings.merge(tables["customers"], on="customer_id", how="left", suffixes=("", "_customer"))
    df = df.merge(tables["drivers"], on="driver_id", how="left", suffixes=("", "_driver"))
    df = df.merge(tables["location"], on=["pickup_location", "drop_location"], how="left")
    df = df.merge(tables["time"], on="hour", how="left")

    if "is_weekend_x" in df.columns:
        df["is_weekend"] = df["is_weekend_x"]
    df = df.drop(columns=[c for c in ("is_weekend_x", "is_weekend_y") if c in df.columns])

    df = df.drop(columns=[c for c in ("customer_name", "driver_name") if c in df.columns])

    return df
```

Four merges in sequence, each a left join (`how="left"`) - every booking row survives every merge even if, say, a particular pickup/drop pair has no matching row in the location-demand table; a left join fills the unmatched columns with `NaN` rather than silently dropping the whole booking. `suffixes=("", "_customer")` on the customers merge means columns that exist in both `bookings` and `customers` keep the bookings version unsuffixed and get `_customer` appended to the customer version - but since `bookings` and `customers` don't actually share any column names besides `customer_id` (the join key, which never gets suffixed), this suffix mostly matters for the *next* merge: `total_completed_rides` exists in both `customers` and `drivers`, so after both merges you end up with `total_completed_rides` (from customers, unsuffixed) and `total_completed_rides_driver` (from drivers, suffixed) - which is exactly the naming pattern `add_reliability_scores()` later depends on. The `is_weekend_x/is_weekend_y` handling exists because *both* `bookings` (via `add_time_parts`) and the `time_features` table have their own `is_weekend` column - merging two DataFrames that share a non-join column name causes pandas to auto-suffix both copies with `_x/_y`. Since `time_features`' copy is a hard-coded placeholder (always 0, per `generate_synthetic_data.py`'s `_make_time_features()`), the real one from bookings (`_x`) is kept and both suffixed copies are dropped, leaving a single clean `is_weekend` column. `customer_name/driver_name` are dropped last - useful for a human reading a table, meaningless (worse, actively unhelpful - a unique-per-row identifier-like string) as a machine learning feature.

#### Lines 65-73 -- `add_fare_features()`

```
def add_fare_features(df):
    df["estimated_fare"] = df["base_fare"] * df["surge_multiplier"]

    df["fare_per_km"] = df["estimated_fare"] / df["distance_km"].replace(0, np.nan)
    df["fare_per_min"] = df["estimated_fare"] / df["trip_duration_min"].replace(0, np.nan)
    df["fare_per_km"] = df["fare_per_km"].replace([np.inf, -np.inf], np.nan).fillna(0)
    df["fare_per_min"] = df["fare_per_min"].replace([np.inf, -np.inf], np.nan).fillna(0)

    return df
```

`estimated_fare` is the fare-prediction model's target - the actual price this booking would cost, given its base fare and surge multiplier. `.replace(0, np.nan)` before dividing is the key defensive move: dividing by a literal `0` distance or duration would produce `inf`, which is both mathematically meaningless here and would corrupt any downstream mean/ median computed from the column; replacing the zero with `NaN` first means the division produces `NaN` instead of `inf`, which then gets explicitly replaced-and-filled to `0` on the next line - a "no ratio

available" sentinel rather than an infinite one.

#### Lines 76-80 -- `add_trip_flags()`

```
def add_trip_flags(df):
    df["long_distance_flag"] = (df["distance_km"] >= df["distance_km"].median()).astype(int)
    df["rush_hour_flag"] = df["is_peak_hour"].fillna(0).astype(int)
    df["city_pair"] = df["pickup_location"].astype(str) + " -> " + df["drop_location"].astype(str)
    return df
```

`long_distance_flag` is relative to *this dataset's own* median distance, not a fixed km threshold - so what counts as "long" adapts automatically if the underlying data changes (a dataset of mostly short intra-city hops would have a much lower "long" threshold than one full of airport runs). `rush_hour_flag` is essentially a renamed copy of `is_peak_hour` (which comes from the `time_features` table) - kept as a separate column because the project spec explicitly names `Rush_Hour_Flag` as a required feature, even though it's redundant with an existing column. `city_pair` is the `City_Pair` feature the spec asks for (`Pickup + Drop`) - built as a readable string ("`Connaught Place -> Karol Bagh`") rather than, say, a numeric hash, so it doubles as something a human can read directly off a dashboard chart, and a tree-based model can still one-hot encode it like any other categorical column.

#### Lines 83-111 -- `add_reliability_scores()`

```
def add_reliability_scores(df):
    df["customer_cancellation_rate"] = (
        df["total_cancelled_rides"]
        / (df["total_completed_rides"] + df["total_cancelled_rides"]).replace(0, np.nan)
    ).fillna(0)

    df["driver_cancellation_rate"] = (
        df["total_cancelled_rides_driver"]
        / (df["total_completed_rides_driver"] + df["total_cancelled_rides_driver"]).replace(0, np.nan)
    ).fillna(0)

    df["driver_reliability_score"] = (
        (df["driver_rating"] * 20) * 0.4
        + (df["acceptance_rate"] * 100) * 0.4
        + ((1 / (1 + df["avg_delay_min"])) * 100) * 0.2
    )

    df["customer_loyalty_score"] = (
        (df["customer_rating"] * 20) * 0.4
        + (df["total_completed_rides"] * 0.4)
        + ((1 - df["customer_cancellation_rate"]) * 100 * 0.2)
    )

    return df
```

The two historical-rate columns follow the same "protect against divide-by-zero, then treat the result as 0 rather than NaN" pattern as `add_fare_features()` - a customer/driver with zero rides on record gets a 0% cancellation rate rather than an undefined one. `driver_reliability_score` and `customer_loyalty_score` are the two composite scores the spec names explicitly (`Driver_Reliability_Score`, `Customer_Loyalty_Score`), each a weighted blend of three underlying signals. The `* 20` on a 1-5 star rating rescales it onto a 0-100 range so it's comparable in magnitude to `acceptance_rate * 100` and the ride-count term before they're weighted and summed - without that rescaling, the rating term would be dwarfed by the others regardless of its assigned weight. `1 / (1 + avg_delay_min)` is a decreasing function of delay that's always between 0 and 1 (a 0-minute average delay gives exactly 1; a large delay pushes it toward 0) - a reasonable way to fold "lower average delay is better" into a formula that otherwise only has terms where "higher is better", without needing a separate sign flip. The 0.4/0.4/0.2 weights are a documented judgment call (see the function's docstring) rather than anything statistically derived - rating and

behavior each count for 40%, and delay/cancellation history for the remaining 20%.

#### Lines 114-119 -- `add_model_targets()`

```
def add_model_targets(df):
    df["ride_outcome_target"] = df["ride_status"]
    df["customer_cancel_flag"] = (df["cancelled_by"] == "Customer").astype(int)
    df["driver_delay_flag"] = (df["driver_delay_min"] >= 10).astype(int)
    return df
```

Three of the four labels `train_model.py` predicts (the fourth, `estimated_fare`, was already created in `add_fare_features()`). `ride_outcome_target` is a literal copy of `ride_status` under a different name - this duplication matters a lot in `train_model.py`, where an earlier bug came from forgetting that `ride_status` itself (not just this renamed copy) needed to be excluded from the ride-outcome model's input features, since the two columns carry identical information. `customer_cancel_flag` and `driver_delay_flag` both threshold a continuous/categorical source column into a binary 0/1 label - 10 minutes is the cutoff chosen for "this counts as a delay", matching what `generate_synthetic_data.py`'s comments describe as the intended threshold.

#### Lines 122-135 -- `final_cleanup()`

```
def final_cleanup(df):
    for col in df.columns:
        is_text = not pd.api.types.is_numeric_dtype(df[col]) and not pd.api.types.is_datetime64_any_dtype(df[col])
        if is_text:
            df[col] = df[col].fillna("Unknown")

    num_cols = df.select_dtypes(include="number").columns
    for col in num_cols:
        df[col] = df[col].replace([np.inf, -np.inf], np.nan).fillna(0)

    return df
```

A second missing-value pass after every merge and derived column - a left join can introduce *fresh* `NaN`s that didn't exist in any single source table (e.g. a pickup/drop pair with no matching row in `location_demand` means `demand_index/zone_type` come back `NaN` for that booking, even though neither `bookings_cleaned.csv` nor `location_demand_cleaned.csv` had a missing value in isolation). The `is_text` check explicitly excludes both numeric *and* datetime columns from the "fill with Unknown" branch - filling a datetime column's `NaN` (properly `NaT`) with the string `"Unknown"` would silently convert that column back to a generic object dtype and break every `.dt.hour`-style accessor used earlier in the pipeline, so datetime columns are deliberately left alone here (any real missing timestamps were already handled by `errors="coerce"` in `add_time_parts()`).

#### Lines 138-160 -- `build_features()` and `main()`

```
def build_features():
    ensure_dirs()
    tables = load_cleaned_tables()

    df = merge_tables(tables)
    df = add_fare_features(df)
    df = add_trip_flags(df)
    df = add_reliability_scores(df)
    df = add_model_targets(df)
    df = final_cleanup(df)

    save_csv(df, "final_merged_data.csv")
    ...
```

The orchestrator, in dependency order: tables have to be merged before any derived feature can reference columns from more than one source table; `add_model_targets()` runs after `add_fare_features()` because `ride_outcome_target` doesn't depend on it but keeping targets together near the end reads more clearly; `final_cleanup()` runs last specifically because it's meant to catch *whatever* NaNs and infinities are left over after every other step, not just the ones from the raw merge.

---

## `src/train_model.py`

Trains all four models, tunes each with `GridSearchCV`, evaluates them, and persists the results. The longest file in the project - walked through function by function below.

### Lines 15-36 -- imports

```
import json
import joblib
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    mean_absolute_error, mean_squared_error, r2_score, roc_auc_score,
)
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler
from xgboost import XGBRegressor
from utils import MODELS_DIR, PROCESSED_DIR, REPORTS_DIR, ensure_dirs, load_csv
```

`json` writes the machine-readable metrics file; `joblib` is scikit-learn's recommended tool for saving/loading a fitted model (handles the numpy arrays and nested objects inside a scikit-learn pipeline more efficiently than Python's generic `pickle` module, though it's pickle-compatible under the hood). Every classifier in this file is a `RandomForestClassifier`; the one regressor is `XGBRegressor` from the separate `xgboost` package, imported on its own line since it isn't part of scikit-learn itself.

### Lines 38-53 -- module-level constants

```
ALWAYS_DROP = [
    "booking_id", "customer_id", "driver_id", "booking_time",
    "customer_name", "driver_name", "is_weekend_x", "is_weekend_y",
]

TARGET_COLUMNS = [
    "ride_status", "ride_outcome_target", "cancelled_by",
    "customer_cancel_flag", "driver_delay_flag",
]

CLASSIFICATION_ACCURACY_BENCHMARK = 0.85
REGRESSION_RMSE_PCT_BENCHMARK = 0.10
```

`ALWAYS_DROP` lists columns no model should ever see as an input feature - identifiers with no predictive meaning (`booking_id`), raw identifiers a model shouldn't memorize (`customer_id/driver_id` - a model that "learns" individual customer IDs wouldn't generalize to a new customer at all), a raw timestamp (superseded by its extracted `hour/day_of_week/ is_weekend` parts), free-text names, and the two duplicate-weekend-column names that shouldn't normally exist post-`feature_engineering.py` but are excluded defensively anyway. `TARGET_COLUMNS` is

defined but - worth noting explicitly - never actually referenced anywhere else in this file; each training function hard-codes its own `extra_drop` list instead of pulling from this constant. It's not wrong, just an unused piece of documentation-as-code that could be deleted or wired in without changing behavior. The two benchmark constants are the specific numbers from the project spec ("85-90% accuracy", "RMSE within +/-10% of actual fare") pulled out as named values so `evaluate_classifier()/evaluate_regressor()` don't have a magic `0.85/0.10` buried in their bodies.

#### Lines 56-72 -- `build_preprocessor()`

```
def build_preprocessor(X):
    num_cols = X.select_dtypes(include="number").columns.tolist()
    cat_cols = [c for c in X.columns if c not in num_cols]

    return ColumnTransformer([
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
        ]), num_cols),
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore")),
        ]), cat_cols),
    ])
```

Shared by all four models via `tune_classifier()/tune_regressor()` below - a change here (say, switching `StandardScaler` for something else) applies everywhere at once rather than needing four separate edits. `num_cols` is found with `select_dtypes(include="number")` (catching every numeric dtype at once); `cat_cols` is defined as *everything else* (`c not in num_cols`) rather than its own `select_dtypes` call - this is deliberately robust to pandas 3.x's `str` dtype for text columns, the same issue that `data_cleaning.py`'s `fill_missing()` had to work around, by never actually checking for a specific "categorical" dtype at all. `ColumnTransformer` applies a different preprocessing pipeline to each column group and concatenates the results: numeric columns get missing values filled with the column median, then are scaled to zero mean/unit variance (important for gradient-based models like `XGBRegressor`, less critical for tree-based `RandomForestClassifier` but harmless either way); categorical columns get missing values filled with the most frequent value in that column, then are one-hot encoded. `handle_unknown="ignore"` on the encoder means a category value seen at prediction time that never appeared during training (a brand-new city, say) doesn't crash the pipeline - it's encoded as all-zeros for that column's one-hot block instead of raising.

#### Lines 75-81 -- `split_xy()`

```
def split_xy(df, target_col, extra_drop):
    drop_cols = set(ALWAYS_DROP) | set(extra_drop) | {target_col}
    X = df.drop(columns=[c for c in drop_cols if c in df.columns])
    y = df[target_col]
    return X, y
```

The shared building block every `train_*_model()` function calls. `set(...) | set(...) | {target_col}` unions three collections of column names into one set (using Python's set union operator, `|`) - using a set rather than concatenating lists means any column that happens to appear in more than one of the three sources (say, a column listed in both `ALWAYS_DROP` and a model's own `extra_drop`) is only counted once, and set membership makes the `if c in df.columns` filter on the next line an O(1) check per column rather than a slower list scan.

#### Lines 84-103 -- `tune_classifier()`

```
def tune_classifier(X_train, y_train):
    pipeline = Pipeline([
        ("preprocessor", build_preprocessor(X_train)),
        ("classifier", RandomForestClassifier(random_state=42, class_weight="balanced")),
    ])
    param_grid = {
        "classifier__n_estimators": [150, 300],
        "classifier__max_depth": [None, 12],
    }
    search = GridSearchCV(pipeline, param_grid, cv=3, scoring="accuracy", n_jobs=-1)
    search.fit(X_train, y_train)
    return search.best_estimator_, search.best_params_
```

`class_weight="balanced"` tells scikit-learn to automatically reweight each class's contribution to the loss function inversely proportional to its frequency - without it, on a dataset where (say) 90% of bookings complete normally, a classifier can score a deceptively high raw accuracy by simply always predicting "Completed" and never learning to recognize the minority classes at all (this is exactly the failure mode described in the comment above this line, and exactly what an earlier version of this project's Customer Cancellation model did before the fix). The `param_grid` keys use scikit-learn's `stepname__parameter` double-underscore syntax - `"classifier__n_estimators"` means "the `n_estimators` parameter of the pipeline step named `classifier`", which is how `GridSearchCV` reaches inside a multi-step `Pipeline` to tune a parameter that belongs to one specific step rather than the pipeline as a whole. Two values for each of two parameters gives 4 total combinations, each evaluated with 3-fold cross-validation (`cv=3`) - 12 model fits total, small enough to finish in seconds on a dataset this size. `n_jobs=-1` tells scikit-learn to use every available CPU core in parallel rather than fitting each combination one at a time. `search.best_estimator_` is the already-refit pipeline using the best-scoring hyperparameters (scikit-learn automatically refits on the *full* training set once the best combination is found, by default); `search.best_params_` is a plain dict of which values won, returned alongside so it can be logged in the metrics report.

#### Lines 106-123 -- `tune_regressor()`

```
def tune_regressor(X_train, y_train):
    pipeline = Pipeline([
        ("preprocessor", build_preprocessor(X_train)),
        ("regressor", XGBRegressor(random_state=42, n_jobs=-1)),
    ])
    param_grid = {
        "regressor__n_estimators": [200, 400],
        "regressor__max_depth": [4, 6],
        "regressor__learning_rate": [0.05, 0.1],
    }
    search = GridSearchCV(pipeline, param_grid, cv=3, scoring="neg_root_mean_squared_error", n_jobs=-1)
    search.fit(X_train, y_train)
    return search.best_estimator_, search.best_params_
```

Same overall shape as `tune_classifier()`, but for the fare model specifically: `XGBRegressor` instead of a `RandomForest` (see the function's docstring for the reasoning - fare is a smooth, roughly-additive target that gradient boosting tends to fit more precisely than bagged trees), and three tuned parameters instead of two, since boosting has an extra `learning_rate` knob that a `RandomForest` doesn't. `scoring= "neg_root_mean_squared_error"` - *negative* because scikit-learn's `GridSearchCV` always maximizes whatever scoring function you give it, and you want to *minimize* RMSE, so scikit-learn's convention is to expose the negated version as the "higher is better" score it can maximize toward. This grid has  $2 \times 2 \times 2 = 8$  combinations  $\times$  3 folds = 24 fits.

#### Lines 126-161 -- `evaluate_classifier()`

```
def evaluate_classifier(name, model, X_test, y_test, class_names):
    preds = model.predict(X_test)
```



```

accuracy = accuracy_score(y_test, preds)

proba = model.predict_proba(X_test)
try:
    if proba.shape[1] == 2:
        auc = roc_auc_score(y_test, proba[:, 1])
    else:
        auc = roc_auc_score(y_test, proba, multi_class="ovr")
except ValueError:
    auc = None
...

```

`model.predict_proba(X_test)` returns a 2D array - one row per test sample, one column per class, holding the model's predicted probability for each class. `proba.shape[1] == 2` checks whether this is a binary classifier (exactly 2 columns) - if so, `roc_auc_score` needs just the probability of the *positive* class (`proba[:, 1]`, i.e. every row's second column). For more than 2 classes (the multiclass Ride Outcome model), the full probability matrix is passed along with `multi_class="ovr"` ("one-vs-rest"), which computes an AUC for each class against all others and averages them - a standard way to extend the binary-only concept of AUC to a multiclass setting. The `try/except ValueError` guards against the rare case where the test split happens to be missing one of the classes entirely (making AUC mathematically undefined for that class) - shouldn't happen given the stratified splits used everywhere in this file, but the code reports "not available" rather than crashing the whole training run if it somehow does.

```

report = classification_report(
    y_test, preds, target_names=class_names, output_dict=True, zero_division=0
)
matrix = confusion_matrix(y_test, preds).tolist()
...
return {
    "model": name,
    "task": "classification",
    "accuracy": round(float(accuracy), 4),
    "auc": round(float(auc), 4) if auc is not None else None,
    "confusion_matrix": matrix,
    "class_names": list(class_names),
    "classification_report": report,
    "meets_benchmark": bool(accuracy >= CLASSIFICATION_ACCURACY_BENCHMARK),
    "benchmark": f">= {CLASSIFICATION_ACCURACY_BENCHMARK:.0%} accuracy",
}

```

`output_dict=True` returns the classification report as a nested Python dict instead of a formatted string - needed so it can be embedded directly into the JSON metrics file rather than stored as unstructured text. `zero_division=0` tells scikit-learn to report `0.0` (rather than raising a warning-triggering division-by-zero) for any class the model never predicts at all - which does happen here for the rarer classes before `class_weight="balanced"` was added, and is a real possibility even after, just less often. `confusion_matrix(...).tolist()` converts scikit-learn's numpy-array result into a plain nested Python list, since numpy arrays aren't JSON-serializable directly. Every numeric value going into the returned dict is explicitly wrapped in `float(...)` or `bool(...)` before rounding - this matters because `accuracy_score` and a comparison like `accuracy >= 0.85` both return numpy scalar types (`numpy.float64`, `numpy.bool_`), and Python's `json.dump` doesn't know how to serialize those natively; casting to the equivalent built-in Python type first is what makes `write_metrics_report()`'s `json.dump()` call work at all without raising a `TypeError`.

#### Lines 164-183 -- `evaluate_regressor()`

```

def evaluate_regressor(name, model, X_test, y_test):
    preds = model.predict(X_test)
    mae = mean_absolute_error(y_test, preds)

```



```

rmse = mean_squared_error(y_test, preds) ** 0.5
r2 = r2_score(y_test, preds)
rmse_pct = rmse / y_test.mean()
...

```

`mean_squared_error(...)` `** 0.5` computes RMSE by hand (square-rooting MSE) rather than calling a dedicated RMSE function, since older scikit-learn versions didn't expose one directly - this works on any version. `rmse_pct = rmse / y_test.mean()` is the number that actually gets checked against the spec's "RMSE within +/-10% of actual fare" benchmark - RMSE alone is in rupees, which isn't directly comparable to a percentage target without first expressing it relative to the typical fare size.

#### Lines 186-210 -- `train_ride_outcome_model()`

```

def train_ride_outcome_model(df):
    X, y = split_xy(
        df, "ride_outcome_target",
        extra_drop=["ride_status", "cancelled_by", "customer_cancel_flag", "driver_delay_flag"],
    )

    label_encoder = LabelEncoder()
    y_encoded = label_encoder.fit_transform(y)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
    )

    model, best_params = tune_classifier(X_train, y_train)
    metrics = evaluate_classifier("Ride Outcome Model", model, X_test, y_test, label_encoder.classes_)
    metrics["best_params"] = best_params

    joblib.dump(model, f"{MODELS_DIR}/ride_outcome_model.pkl")
    joblib.dump(label_encoder, f"{MODELS_DIR}/ride_outcome_label_encoder.pkl")
    return metrics

```

The comment right above the `extra_drop` list (in the actual file) explains the single most important line here: `"ride_status"` and `"cancelled_by"` *must* be in this drop list, because `ride_outcome_target` is a direct unmodified copy of `ride_status` (from `add_model_targets()` in `feature_engineering.py`) - leaving `ride_status` in the input features would let the model "predict" the target by just reading its own answer back out of a different column, which is exactly what an earlier version of this function did, producing a suspiciously perfect 100% test accuracy that turned out to be pure data leakage rather than real predictive skill. `LabelEncoder` converts the three text labels (`"Completed"`, `"Cancelled"`, `"Incomplete"`) into integers (0, 1, 2) - scikit-learn's classifiers work with either, but encoding explicitly here means the *encoder itself* can be saved and reused later to convert a raw integer prediction back into a human-readable label (that's exactly what `ride_outcome_label_encoder.pkl` is for, and why `predict.py` and `streamlit_app.py` both load it alongside the model). `stratify=y_encoded` on the train/test split ensures the Completed/Cancelled/Incomplete proportions in the test set match the full dataset's proportions as closely as possible, rather than a plain random split potentially leaving the test set with, say, zero Incomplete examples by chance - critical for a dataset where one class (Completed) already dominates.

#### Lines 213-227 -- `train_fare_model()`

```

def train_fare_model(df):
    X, y = split_xy(
        df, "estimated_fare",
        extra_drop=["ride_status", "ride_outcome_target", "cancelled_by",
                    "customer_cancel_flag", "driver_delay_flag"],
    )

```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model, best_params = tune_regressor(X_train, y_train)
metrics = evaluate_regressor("Fare Prediction Model", model, X_test, y_test)
metrics["best_params"] = best_params

joblib.dump(model, f"{MODELS_DIR}/fare_model.pkl")
return metrics

```

No `stratify=` argument here, since stratification is a classification-only concept (it works by preserving *class* proportions) - a regression target like `estimated_fare` is continuous, so a plain random split is the right tool. The `extra_drop` list excludes every ride-outcome/cancellation- related column, not because they'd leak the fare answer directly, but because they're outcomes that wouldn't actually be known yet at the moment you'd want to predict a fare (before the trip even starts) - keeping them out keeps the model honest about what information is realistically available at prediction time.

#### Lines 230-247 -- `train_customer_cancel_model()`

```

def train_customer_cancel_model(df):
    X, y = split_xy(
        df, "customer_cancel_flag",
        extra_drop=["ride_status", "ride_outcome_target", "cancelled_by", "driver_delay_flag"],
    )
    ...
    metrics = evaluate_classifier(
        "Customer Cancellation Risk Model", model, X_test, y_test, ["No Cancel", "Cancel"]
    )
    ...

```

Same overall shape as the ride-outcome model, but binary (`customer_cancel_flag` is 0/1) instead of 3-class, so there's no `LabelEncoder` needed here - the class names `["No Cancel", "Cancel"]` are passed straight to `evaluate_classifier()` as plain strings for the confusion-matrix/classification-report labels, since 0/1 already means something numerically without needing to be encoded/decoded.

#### Lines 250-275 -- `train_driver_delay_model()`

```

def train_driver_delay_model(df):
    """The model the spec asked for that was never actually built - ..."""
    X, y = split_xy(
        df, "driver_delay_flag",
        extra_drop=["ride_status", "ride_outcome_target", "cancelled_by",
                    "customer_cancel_flag", "driver_delay_min"],
    )
    ...

```

The fourth model, added to close the gap the original project audit found - the target column itself (`driver_delay_flag`) already existed in `feature_engineering.py`, but nothing ever trained on it before this function existed. The one thing to notice in its `extra_drop` list that the other three functions don't need: `"driver_delay_min"` - the raw continuous delay value the binary flag is thresholded from (`driver_delay_min >= 10`, back in `add_model_targets()`). Leaving that raw column in the input features would be exactly the same kind of leakage bug the ride-outcome model had - the model could trivially "learn" the threshold rule instead of the actual underlying reliability signals (driver rating, acceptance rate, current traffic) the model is supposed to generalize from.

#### Lines 278-307 -- `write_metrics_report()`

```
def write_metrics_report(all_metrics):
    with open(f"{REPORTS_DIR}/model_metrics.json", "w", encoding="utf-8") as f:
        json.dump(all_metrics, f, indent=2)

    lines = ["# Model Evaluation Report", ""]
    for metrics in all_metrics:
        lines.append(f"## {metrics['model']}")
        ...
        status = "MEETS" if metrics["meets_benchmark"] else "BELOW"
        lines.append(f"- Benchmark ({metrics['benchmark']}): **{status}**")
        lines.append("")

    with open(f"{REPORTS_DIR}/MODEL_EVALUATION.md", "w", encoding="utf-8") as f:
        f.write("\n".join(lines))
    ...
```

Writes the same underlying metrics twice, in two different formats for two different audiences: `model_metrics.json` is machine-readable (the Streamlit dashboard's Dashboard page loads this directly to show live model-performance tiles), while `MODEL_EVALUATION.md` builds a list of Markdown lines and joins them with newlines into one human-readable document. The `if metrics["task"] == "classification": ... else: ...` branch inside the loop formats each model's section differently depending on whether it's a classifier (accuracy/AUC/confusion matrix) or the one regressor (MAE/RMSE/R2) - the two task types don't share the same metrics, so this file can't format them identically.

#### Lines 310-320 -- `main()`

```
def main():
    ensure_dirs()
    df = load_csv("final_merged_data.csv", folder=PROCESSED_DIR)

    all_metrics = [
        train_ride_outcome_model(df),
        train_fare_model(df),
        train_customer_cancel_model(df),
        train_driver_delay_model(df),
    ]
    write_metrics_report(all_metrics)
```

Trains all four models against the exact same source DataFrame (each `train_*_model()` function does its own `split_xy()` call internally, so there's no risk of one model's preprocessing leaking into another's), then writes one combined report covering all four at once.

---

#### `src/predict.py`

A thin loading/inference layer over the four saved models, plus a self-test when run directly.

#### Lines 19-26 -- `_load_model()`

```
def _load_model(filename):
    path = f"{MODELS_DIR}/{filename}"
    try:
        return joblib.load(path)
    except FileNotFoundError as exc:
        raise FileNotFoundError(
            f"Could not find {path}. Run src/train_model.py first."
        ) from exc
```

The leading underscore in `_load_model` is a Python convention signaling "internal helper, not meant to be imported/called from outside this module" - every other function in this file is a public `predict_*` function meant to be imported by `streamlit_app.py`, but this one is only ever called from within `predict.py` itself. Same catch-and-re-raise-with-context pattern as `utils.load_csv()`.

### Lines 29-53 -- the four `predict_*` functions

```
def predict_ride_outcome(input_df):
    model = _load_model("ride_outcome_model.pkl")
    label_encoder = _load_model("ride_outcome_label_encoder.pkl")
    pred = model.predict(input_df)
    return label_encoder.inverse_transform(pred)[0]

def predict_fare(input_df):
    model = _load_model("fare_model.pkl")
    pred = model.predict(input_df)
    return round(float(pred[0]), 2)

def predict_customer_cancel_risk(input_df):
    model = _load_model("customer_cancel_model.pkl")
    pred = model.predict(input_df)
    proba = model.predict_proba(input_df)[0][1]
    return int(pred[0]), round(float(proba), 4)

def predict_driver_delay_risk(input_df):
    model = _load_model("driver_delay_model.pkl")
    pred = model.predict(input_df)
    proba = model.predict_proba(input_df)[0][1]
    return int(pred[0]), round(float(proba), 4)
```

Every function reloads its model from disk on every call rather than caching it - fine for a manual smoke test or an occasional call, though `streamlit_app.py` deliberately does its own model loading with `@st.cache_resource` instead of calling these functions directly, so the dashboard doesn't reload a model file from disk on every single button click. `model.predict(input_df)` returns a numpy array even for a single-row input, hence `[0]` to pull out the one prediction each function cares about. `predict_ride_outcome`'s result needs `label_encoder.inverse_transform(...)` to turn the model's raw integer output (0/1/2) back into the original text label ("`Completed`" / "`Cancelled`" / "`Incomplete`") - the encoder saved alongside the model in `train_ride_outcome_model()` is exactly what makes that reversal possible. The two binary-risk functions return a `(prediction, probability)` tuple instead of just the 0/1 prediction, since "how confident is the model" (the probability) is arguably more useful to show a user than a bare yes/no.

### Lines 56-75 -- the `_main` self-test

```
if __name__ == "__main__":
    df = load_csv("final_merged_data.csv", folder=PROCESSED_DIR)
    sample = df.iloc[[0]].copy()

    common_drop = [...]
    fare_drop = common_drop + ["estimated_fare"]
    delay_drop = common_drop + ["driver_delay_min"]

    def _prep(drop_cols):
        return sample.drop(columns=[c for c in drop_cols if c in sample.columns])

    print("Ride outcome:", predict_ride_outcome(_prep(common_drop)))
    print("Fare:", predict_fare(_prep(fare_drop)))
```

```
print("Customer cancel risk:", predict_customer_cancel_risk(_prep(common_drop)))
print("Driver delay risk:", predict_driver_delay_risk(_prep(delay_drop)))
```

`df.iloc[[0]]` (double brackets) selects the first row as a *DataFrame* with one row, not a *Series* - scikit-learn's `.predict()` expects a 2D *DataFrame* shaped like the training data, and `df.iloc[0]` (single brackets) would instead return a 1D *Series*, which doesn't have the right shape. `common_drop` is the same list of leakage/identifier columns every model needs excluded regardless of which one you're calling (the comment above it explains why: none of the four raw target representations are legitimate features for *any* of the four models, so there's no need for four separately-maintained near-identical drop lists). `fare_drop` and `delay_drop` each extend that shared base with the one extra column specific to their own model (`estimated_fare` for itself, `driver_delay_min` as the leakage source for the delay flag). `_prep` is a small local closure defined inline (not a module-level function, since it only makes sense in the context of this one `sample` row) that applies whichever drop list is passed in, filtered down to columns that actually exist in `sample` first. Running this file directly (`python src/predict.py`) is the "demo walkthrough" referenced in the README - a quick sanity check that all four models load and produce plausible output against a real row.

---

### `src/load_db.py`

Loads the cleaned CSVs into a local SQLite database built from `db/schema.sql`.

### Lines 18-19 -- path constants

```
DB_PATH = f"{DB_DIR}/rapido.db"
SCHEMA_PATH = f"{DB_DIR}/schema.sql"
```

Both built from `DB_DIR` (imported from `utils.py`), so this script's output location moves automatically if `DB_DIR` is ever redefined.

### Lines 22-47 -- `TABLE_SOURCES`

```
TABLE_SOURCES = [
    ("customers", "customers_cleaned.csv", [
        "customer_id", "customer_name", "customer_rating",
        "total_completed_rides", "total_cancelled_rides",
        "avg_monthly_spend", "preferred_vehicle",
    ]),
    ("drivers", "drivers_cleaned.csv", [...]),
    ("location_demand", "location_demand_cleaned.csv", [...]),
    ("time_features", "time_features_cleaned.csv", [...]),
    ("bookings", "bookings_cleaned.csv", [...]),
]
```

A list of `(table_name, source_csv, columns)` tuples, in this specific order deliberately - `customers`, `drivers`, `location_demand`, and `time_features` all come *before* `bookings`, because `bookings` has foreign keys pointing at the first three (per `db/schema.sql`), and SQLite enforces those foreign keys at insert time once `PRAGMA foreign_keys = ON` is set - inserting a booking that references a `customer_id` which doesn't exist in the `customers` table yet would fail. The explicit `columns` list for each table matters because `bookings_cleaned.csv` (and the others) carries extra derived columns from `data_cleaning.py` that the SQL schema was never designed to hold - this list keeps only the columns `db/schema.sql` actually defines, in the order that table's `CREATE TABLE` statement expects.

### Lines 50-56 -- `build_schema()`

```
def build_schema(conn):
    with open(SCHEMA_PATH, "r", encoding="utf-8") as f:
        schema_sql = f.read()
    try:
        conn.executescript(schema_sql)
    except sqlite3.Error as exc:
        raise RuntimeError(f"Failed to apply {SCHEMA_PATH}: {exc}") from exc
```

`conn.executescript(...)` runs an entire multi-statement SQL file in one call (SQLite's regular `conn.execute()` only accepts a single statement) - exactly what's needed to run every `CREATE TABLE/CREATE INDEX` statement in `schema.sql` sequentially. Catching `sqlite3.Error` specifically (the base class for every SQLite-related exception) and re-raising as a `RuntimeError` with the schema file path attached makes a syntax error in the DDL file immediately traceable to "the schema file, specifically" rather than a bare SQLite error message with no context about which file caused it.

### Lines 59-69 -- `load_table()`

```
def load_table(conn, table_name, csv_filename, columns):
    df = load_csv(csv_filename, folder=PROCESSED_DIR)[columns]
    try:
        conn.execute(f"DELETE FROM {table_name}")
        df.to_sql(table_name, conn, if_exists="append", index=False)
    except sqlite3.Error as exc:
        raise RuntimeError(f"Failed to load {csv_filename} into {table_name}: {exc}") from exc
    print(f"Loaded {len(df):,} rows into {table_name}")
```

`load_csv(...)[columns]` reads the full cleaned CSV, then immediately selects only the columns this table's schema expects, in that exact order - pandas' `[list_of_columns]` indexing both filters and reorders in one step. `conn.execute(f"DELETE FROM {table_name}")` clears out any existing rows in that table *before* inserting - this is what makes re-running `load_db.py` idempotent: run it once, run it again after regenerating the data, and the table ends up with exactly the current CSV's rows, not the current rows appended on top of a stale previous load. `df.to_sql(table_name, conn, if_exists="append", index=False)` is pandas' built-in bulk-insert helper - `if_exists="append"` here doesn't conflict with the `DELETE` just above; it just tells `to_sql` not to try to `DROP`/recreate the table itself (which would lose the schema's constraints and indexes), only to insert rows into whatever table already exists. `index=False` again skips writing pandas' row index as a spurious extra column, same reasoning as `utils.save_csv()`.

### Lines 72-95 -- `main()`

```
def main():
    ensure_dirs()
    conn = sqlite3.connect(DB_PATH)
    conn.execute("PRAGMA foreign_keys = ON")
    try:
        build_schema(conn)
        for table_name, csv_filename, columns in TABLE_SOURCES:
            load_table(conn, table_name, csv_filename, columns)
        conn.commit()
        print(f"\nDatabase ready: {DB_PATH}")
    except (RuntimeError, sqlite3.Error) as exc:
        conn.rollback()
        print(f"Load failed, rolled back: {exc}")
        raise
    finally:
        conn.close()
```

`sqlite3.connect(DB_PATH)` creates the database file if it doesn't already exist, or opens the existing one. `conn.execute("PRAGMA foreign_keys = ON")` matters because SQLite, unusually among databases, does *not* enforce foreign-key constraints by default - without this line, the foreign keys declared in `schema.sql` would exist as documentation only, silently allowing a booking to reference a nonexistent `customer_id`. The `try/except/finally` structure is real transaction handling: if anything inside the `try` block raises (a bad CSV, a foreign-key violation, a malformed schema), `conn.rollback()` undoes every change made in this run before the exception is re-raised (the bare `raise` at the end re-raises the *same* exception that was just caught, after the rollback has run) - so a failed load never leaves the database in a half-loaded, inconsistent state. `conn.close()` in the `finally` block runs no matter what happened above it, ensuring the database connection is always released.

---

## `src/eda.py`

Generates the EDA charts and the written report, mirroring the same "function per chart, orchestrator at the bottom" structure as `train_model.py`.

### Lines 28-39 -- configuration

```
CHART_DIR = f"{REPORTS_DIR}/eda_charts"
REPORT_FILE = f"{REPORTS_DIR}/EDA_REPORT.md"

BLUE = "#2a78d6"
CATEGORICAL = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#4a3aa7"]
BLUE_CMAP = sns.light_palette(BLUE, as_cmap=True)

WEEKDAY_ORDER = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]

sns.set_theme(style="white", rc={"axes.grid": False})
```

The same `BLUE/CATEGORICAL` hex palette used in `app/streamlit_app.py`, kept in sync between the two files on purpose - a chart in the static report and the equivalent chart in the live dashboard use identical colors. `sns.light_palette(BLUE, as_cmap=True)` builds a single-hue *sequential* colormap (light-to-dark blue) from that one blue - used for the cancellation heatmap, where color needs to represent a continuous magnitude (cancellation rate), not a category, so a single-hue ramp is the right choice rather than the multi-color `CATEGORICAL` list. `WEEKDAY_ORDER` exists because pandas' `.value_counts()` on a weekday-name column would otherwise sort alphabetically (Friday, Monday, Saturday, ...) - `.reindex(WEEKDAY_ORDER)` in `chart_ride_volume()` re-sorts the result into calendar order instead. `sns.set_theme(style="white", ...)` sets a plain white background with gridlines explicitly turned off (`rc={"axes.grid": False}`) as the default for every figure created for the rest of the script's run.

### Lines 46-72 -- `chart_ride_volume()`

```
def chart_ride_volume(df):
    fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))

    by_hour = df.groupby("hour").size()
    axes[0].bar(by_hour.index, by_hour.values, color=BLUE)
    ...

    by_weekday = df["day_of_week"].value_counts().reindex(WEEKDAY_ORDER)
    axes[1].bar(by_weekday.index, by_weekday.values, color=CATEGORICAL[1])
    axes[1].tick_params(axis="x", rotation=45)

    by_city = df["city"].value_counts()
    axes[2].barh(by_city.index[::-1], by_city.values[::-1], color=CATEGORICAL[2])
    ...
```



`plt.subplots(1, 3, ...)` creates one figure with three side-by-side subplot axes in a single row, returned as an array (`axes`) indexed 0-2 - answering all three "ride volume by X" spec questions in one combined figure rather than three separate images, since they're the same underlying question at three granularities.

`axes[2].barh(by_city.index[::-1], by_city.values[::-1], ...)` - the `[::-1]` reverses both the city names and their counts in lockstep. This matters because `.value_counts()` already sorts descending (busiest city first), and matplotlib's `barh` draws bars bottom-to-top; without the reversal, the busiest city would render at the *bottom* of the chart, which reads backwards compared to how a ranked list normally reads top-to-bottom - reversing both arrays together puts the busiest city visually on top instead.

#### Lines 75-96 -- `chart_cancellation_heatmap()`

```
def chart_cancellation_heatmap(df):
    rate_table = (
        df.assign(is_cancelled=(df["ride_status"] == "Cancelled").astype(int))
        .pivot_table(index="city", columns="time_bucket", values="is_cancelled", aggfunc="mean")
        * 100
    )
    bucket_order = ["Morning", "Late Morning", "Afternoon", "Evening", "Night"]
    rate_table = rate_table[[c for c in bucket_order if c in rate_table.columns]]

    fig, ax = plt.subplots(figsize=(8, 5))
    sns.heatmap(rate_table, annot=True, fmt=".1f", cmap=BLUE_CMAP, ax=ax, cbar_kws={"label": "Cancellation
    ...
```

`df.assign(is_cancelled=...)` creates a temporary boolean-as-int column inline without mutating `df` itself (`.assign()` returns a new DataFrame rather than modifying in place, unlike `df["col"] = ...`) - useful here since this 0/1 column only exists to make the next line's aggregation work and has no reason to persist on the original DataFrame. `.pivot_table(index="city", columns="time_bucket", values="is_cancelled", aggfunc="mean")` is the single line doing the real work: it groups by every combination of city and time bucket, and for each combination takes the *mean* of the 0/1 `is_cancelled` column - which, for a column of 0s and 1s, is exactly the cancellation rate for that city/time-bucket combination. Multiplying by 100 converts the 0-1 proportion into a percentage. The next two lines re-order the resulting table's columns into calendar-ish order (Morning through Night) rather than whatever order `pivot_table` happened to produce them in, guarded by `if c in rate_table.columns` in case a bucket happens to be entirely absent from the data. `sns.heatmap(..., annot=True, fmt=".1f")` draws the grid with each cell's actual percentage value printed inside it (`annot=True`), formatted to one decimal place - so the chart works both as a quick visual scan (darker = higher cancellation rate, from the single-hue `BLUE_CMAP`) and as a precise reference table at the same time.

#### Lines 99-114 -- `chart_distance_vs_fare()`

```
def chart_distance_vs_fare(df):
    fig, ax = plt.subplots(figsize=(7, 5.5))
    for i, (vehicle, group) in enumerate(df.groupby("vehicle_type")):
        ax.scatter(group["distance_km"], group["estimated_fare"], s=10, alpha=0.4,
                   color=CATEGORICAL[i % len(CATEGORICAL)], label=vehicle)
    ...
    return df["distance_km"].corr(df["estimated_fare"])
```

`df.groupby("vehicle_type")` iterated directly in a `for` loop yields (`group_key, group_dataframe`) pairs one at a time - here, one scatter layer is drawn per vehicle type, each in its own color from the shared `CATEGORICAL` palette, rather than one undifferentiated scatter of every point in a single color. `alpha=0.4` makes each point partially transparent, so in dense regions where many points overlap, the plot visually darkens rather than becoming one solid blob - a cheap way to show point density without a separate density-estimation step. `i % len(CATEGORICAL)` wraps the color index back to 0 if there were ever more vehicle types than colors in the palette (there are only 3 vehicle types against 6 palette colors here, so this never actually wraps in practice, but it's a safe



guard against an `IndexError` if that ever changed). `df["distance_km"].corr(df["estimated_fare"])` computes the Pearson correlation coefficient between the two columns directly via pandas' built-in `.corr()` method - returned from the function (rather than baked into the chart itself) so `build_report()` can quote the exact number in the written text.

#### Lines 117-129 -- `chart_rating_distribution()`

Two side-by-side histograms (customer ratings, driver ratings), same `plt.subplots(1, 2, ...)` pattern as `chart_ride_volume()` but with only two panels instead of three. `bins=15` splits the 1.0-5.0 rating range into 15 equal-width buckets for the histogram - enough resolution to show the distribution's shape without being so fine-grained that individual bars become noisy and hard to read.

#### Lines 132-147 -- `chart_customer_vs_driver_cancellation()`

```
def chart_customer_vs_driver_cancellation(df):
    cancelled = df[df["ride_status"] == "Cancelled"]
    counts = cancelled["cancelled_by"].value_counts()
    ...
```

Filters down to *only* the cancelled rides first (`cancelled_by` is meaningless - always `"None"` - for a completed or incomplete ride), then counts how many of those cancellations were attributed to the customer vs. the driver. This is the "customer vs. driver behavior comparison" the spec asks for, interpreted specifically as "of the rides that got cancelled, who cancelled them" rather than two unrelated customer-only and driver-only charts.

#### Lines 150-160 -- `chart_payment_methods()`

A straightforward bar chart of `payment_method` value counts - the simplest chart function in the file, included mainly to show the pattern holds even for the least involved of the seven required charts.

#### Lines 163-183 -- `chart_traffic_weather_vs_cancellation()`

```
def chart_traffic_weather_vs_cancellation(df):
    fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

    is_cancelled = (df["ride_status"] == "Cancelled").astype(int)
    by_traffic = df.assign(is_cancelled=is_cancelled).groupby("traffic_level")["is_cancelled"].mean() * 100
    by_traffic = by_traffic.reindex(["Low", "Medium", "High"])
    axes[0].bar(by_traffic.index, by_traffic.values, color=CATEGORICAL[3])
    ...

    by_weather = df.assign(is_cancelled=is_cancelled).groupby("weather_condition")["is_cancelled"].mean()
    axes[1].bar(by_weather.index, by_weather.values, color=CATEGORICAL[4])
    ...
```

Same "mean of a 0/1 column = rate" trick as the cancellation heatmap, applied to two separate single-dimension breakdowns instead of one two-dimensional pivot table. `by_traffic.reindex(["Low", "Medium", "High"])` forces a logical low-to-high ordering on the x-axis (rather than whatever order `groupby` happened to produce, likely alphabetical - "High, Low, Medium" would read oddly). `by_weather`, by contrast, is sorted by `.sort_values(ascending=False)` - worst weather condition first - since there's no single natural ordering for "Clear/Cloudy/Rain/Smog" the way there is for Low/Medium/High, so ranking by the actual cancellation rate is more informative than any arbitrary fixed order would be.

#### Lines 186-259 -- `build_report()`

Same overall approach as `train_model.py`'s `write_metrics_report()` - builds a `lines` list of Markdown strings and joins them into one file at the end. Every number quoted in the text (peak hour, busiest city, correlation coefficient, mean ratings, cancellation counts by who cancelled, most-used payment method, worst traffic/weather conditions) is pulled from the actual return values of the chart functions above, rather than hard-coded - so re-running `eda.py` after the underlying data changes regenerates a report that's still accurate, not stale text sitting next to fresh charts.

### Lines 262-279 -- `run_eda()` and entry point

```
def run_eda():
    os.makedirs(CHART_DIR, exist_ok=True)
    df = load_data()

    by_hour, by_weekday, by_city = chart_ride_volume(df)
    chart_cancellation_heatmap(df)
    distance_fare_corr = chart_distance_vs_fare(df)
    chart_rating_distribution(df)
    cancel_by_who = chart_customer_vs_driver_cancellation(df)
    payment_counts = chart_payment_methods(df)
    by_traffic, by_weather = chart_traffic_weather_vs_cancellation(df)

    build_report(df, by_hour, by_city, distance_fare_corr, cancel_by_who, payment_counts, by_traffic, by_weather)
    ...
```

The orchestrator - each chart function is called exactly once; the ones whose return values `build_report()` actually needs (`by_hour`, `by_city`, `distance_fare_corr`, `cancel_by_who`, `payment_counts`, `by_traffic`, `by_weather`) are captured into variables, while `chart_cancellation_heatmap(df)` and `chart_rating_distribution(df)` are called without keeping their return values, since the written report only references those two charts by image, not by any specific number pulled from them.

---

### `app/streamlit_app.py`

The interactive dashboard. Streamlit re-runs this entire script top to bottom on every user interaction (every filter change, every button click) - that execution model is why expensive work (loading the data, loading the models) is wrapped in caching decorators, and why the four prediction pages build a fresh input row from scratch on every button press rather than trying to persist state across reruns manually.

### Lines 16-22 -- module-level constants

```
CATEGORICAL_COLORS = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#4a3aa7"]

PREDICTION_LEAK_COLUMNS = [
    "ride_status", "ride_outcome_target", "cancelled_by",
    "customer_cancel_flag", "driver_delay_flag",
    "booking_id", "customer_id", "driver_id", "booking_time",
]
```

`CATEGORICAL_COLORS` matches `eda.py`'s palette exactly, same reasoning as before. `PREDICTION_LEAK_COLUMNS` is this file's equivalent of `train_model.py`'s `ALWAYS_DROP` - every column that's either a raw target representation or a bare identifier, dropped before *any* model here makes a prediction, regardless of which of the four models is being called.

### Lines 27-49 -- the three cached loaders

```

@st.cache_data
def load_data():
    return pd.read_csv("data/processed/final_merged_data.csv")

@st.cache_resource
def load_models():
    return {
        "ride_outcome": joblib.load("models/ride_outcome_model.pkl"),
        ...
    }

@st.cache_data
def load_metrics():
    try:
        with open("reports/model_metrics.json", "r", encoding="utf-8") as f:
            return json.load(f)
    except FileNotFoundError:
        return None

```

`@st.cache_data` caches a function's *return value* by hashing its arguments and re-running only if the arguments (or the function's own source code) change - appropriate for `load_data()` and `load_metrics()`, which return plain data (a DataFrame, a dict/list from JSON). `@st.cache_resource` is a different decorator meant specifically for objects that shouldn't be copied or re-serialized on every cache hit - like loaded scikit-learn models, which Streamlit's docs specifically recommend this decorator for over `cache_data`, since model objects can be large and `cache_resource` returns the *same* object reference on every call rather than a copy. Both decorators mean each of these three functions' bodies only actually execute once per Streamlit session, no matter how many times the surrounding script reruns as the user clicks around. `load_metrics()` returns `None` (rather than raising) if the file is missing - deliberately softer error handling than `load_data()/load_models()`, since the dashboard should still work perfectly well without the metrics file, just skip the "Model Performance" section (a real design choice, not neglect - see the Dashboard page code below).

#### Lines 52-66 -- `build_template_row()` and `prepare_for_prediction()`

```

def build_template_row(df, overrides):
    row = df.iloc[[0]].copy()
    for col, value in overrides.items():
        row[col] = value
    return row

def prepare_for_prediction(row, extra_drop=None):
    drop_cols = list(PREDICTION_LEAK_COLUMNS) + list(extra_drop or [])
    return row.drop(columns=[c for c in drop_cols if c in row.columns])

```

The single most important design decision in this file, and the fix for what was previously the most fragile part of the dashboard: rather than hand-typing a dictionary with all ~50 column names and values the model expects (which has to be kept in perfect sync with `feature_engineering.py` by hand, and silently breaks the moment a column is renamed or added), `build_template_row()` takes the *first real booking* from the processed dataset as a starting point (`df.iloc[[0]].copy()` - double brackets for a one-row DataFrame, `.copy()` so mutating it doesn't touch the cached `df` itself) and then only overwrites the specific columns passed in via the `overrides` dict - everything the user *didn't* touch in the UI keeps a real, valid value from an actual booking. `for col, value in overrides.items(): row[col] = value` loops over the dict and sets each column one at a time. `prepare_for_prediction()` then strips out the leakage/identifier columns (plus, optionally, one or two extra columns specific to the model being called, via `extra_drop`) right before handing the row to a model's `.predict()` - the exact same drop-column philosophy as `train_model.py`'s `split_xy()`, just applied to a single hand-built row instead of a full training DataFrame.

## Lines 69-77 -- startup: load everything, handle failure

```
try:
    df = load_data()
    models = load_models()
except FileNotFoundError as exc:
    st.error(f"Missing a required file: {exc}")
    st.error("Run the pipeline first: data_cleaning.py -> feature_engineering.py -> train_model.py")
    st.stop()

metrics = load_metrics()
```

This code runs at module level (not inside a function) - it executes immediately every time Streamlit reruns the script. If either the processed data or any of the five model/encoder files is missing, `st.error(...)` renders a red error box in the browser with a specific, actionable message, and `st.stop()` halts the rest of the script immediately - nothing below this block runs, so the user sees a clear error instead of a wall of secondary tracebacks from every page trying to use a `df` or `models` that was never successfully loaded.

## Lines 87-113 -- the Dashboard page

```
if page == "Dashboard":
    st.subheader("Overview")
    col1, col2, col3 = st.columns(3)
    col1.metric("Total Rides", f"{len(df):,}")
    col2.metric("Completed Rides", f"{int((df['ride_status'] == 'Completed').sum()):,}")
    col3.metric("Cancellation Rate", f"{(df['ride_status'].eq('Cancelled').mean() * 100):.1f}%")

    st.subheader("Ride Volume by Hour")
    st.line_chart(df.groupby("hour").size())

    st.subheader("Ride Status Distribution")
    st.bar_chart(df["ride_status"].value_counts())

    if metrics:
        st.subheader("Model Performance (from reports/model_metrics.json)")
        cols = st.columns(4)
        for col, m in zip(cols, metrics):
            with col:
                if m["task"] == "classification":
                    st.metric(m["model"], f"{m['accuracy']:.1%}", "accuracy")
                    st.caption(f"AUC: {m['auc']:.3f}" if m["auc"] is not None else "AUC: n/a")
                else:
                    st.metric(m["model"], f"{m['rmse']:.2f}", "RMSE")
                    st.caption(f"{m['rmse_pct_of_mean']:.1%} of mean fare, R2={m['r2']:.3f}")
                    st.caption("[x] meets benchmark" if m["meets_benchmark"] else "[!] below benchmark")
            else:
                st.info("Run `python src/train_model.py` to generate a model performance summary.")
```

`st.columns(3)` (and later `st.columns(4)`) create side-by-side layout slots, same pattern as the traffic-insight dashboard's summary tiles. `(df['ride_status'] == 'Completed').sum()` sums a boolean Series, which counts `True` as 1 - a compact way to count matching rows without a separate `.value_counts()` call. `zip(cols, metrics)` pairs each of the four layout columns with one model's metrics dict, in whatever order `train_model.py` originally wrote them to the JSON file (ride outcome, fare, customer cancellation, driver delay) - `with col:` inside the loop routes every `st.metric/st.caption` call for that iteration into that specific column rather than the main page body. The `if m["task"] == "classification": ... else: ...` branch mirrors `write_metrics_report()`'s same branch in `train_model.py`, since a classifier's metrics dict has different keys (`accuracy`, `auc`) than the regressor's (`rmse`, `r2`). The `if metrics: ... else: ...` at the outer level is what makes this whole section optional and non-fatal - if `train_model.py` hasn't been run yet (so `reports/model_metrics.json` doesn't exist), `load_metrics()` returned `None` earlier, and the dashboard just

shows a helpful info message here instead of crashing.

## Lines 115-126 -- the EDA page

A lightweight interactive city filter - select a city, see that city's hour/vehicle-type/payment-method breakdowns as three side-by-side mini bar charts. Explicitly captioned as a companion to, not a replacement for, the full [reports/EDA\\_REPORT.md](#) write-up - this page exists for quick "let me just check one city" exploration, not comprehensive analysis.

## Lines 128-160 -- the Ride Outcome Prediction page

```
elif page == "Ride Outcome Prediction":
    st.subheader("Predict Ride Outcome")

    col1, col2 = st.columns(2)
    with col1:
        city = st.selectbox("City", sorted(df["city"].dropna().unique()))
        ...
    with col2:
        hour = st.slider("Hour", 0, 23, 9)
        ...

    if st.button("Predict Ride Outcome"):
        row = build_template_row(df, {
            "city": city, "vehicle_type": vehicle_type, ...,
            "is_peak_hour": int(hour in (7, 8, 9, 17, 18, 19)),
            "rush_hour_flag": int(hour in (7, 8, 9, 17, 18, 19)),
            ...
        })
        input_df = prepare_for_prediction(row)
        pred = models["ride_outcome"].predict(input_df)
        label = models["ride_outcome_encoder"].inverse_transform(pred)[0]
        st.success(f"Predicted Ride Outcome: **{label}**")
```

Every widget (`st.selectbox`, `st.number_input`, `st.slider`) is assigned to a local variable up front, laid out across two columns purely for visual balance - none of that runs any prediction yet, it just collects the user's current choices. The actual prediction only happens inside `if st.button("Predict Ride Outcome")`: - Streamlit re-runs the whole script on every widget change, but the button's own state (was it *just* clicked this rerun) is what gates whether the block underneath actually executes, so idly adjusting a slider doesn't fire a prediction on every tiny movement. `build_template_row(df, {...})` is where the "start from a real row, override what the user controls" pattern from earlier actually gets used - notice `is_peak_hour/rush_hour_flag` are both recomputed here from the user's chosen `hour`, using the same `(7, 8, 9, 17, 18, 19)` peak-hour tuple that `generate_synthetic_data.py` uses when building the original training data - if that tuple keeps drifting apart between the two files over time, the model would start seeing inconsistent peak-hour labeling between training and prediction, so keeping the two in sync (even though they're not literally shared via a single constant) matters for correctness.

## Lines 162-181 -- the Fare Prediction page

Same "collect widgets, build template row, drop leak columns, predict" shape as the ride-outcome page, but this time `prepare_for_prediction(row, extra_drop=["estimated_fare"])` passes an `extra_drop` - unlike the ride-outcome page, this page's target (`estimated_fare`) is *not* already covered by `PREDICTION_LEAK_COLUMNS`, since fare prediction is the one model where `estimated_fare` genuinely needs to be excluded as a feature but every other model is allowed to see it. Notice the overrides dict recomputes `fare_per_km/fare_per_min` from the user's own inputs `((base_fare * surge_multiplier) / max(distance_km, 0.1))` rather than leaving those two engineered-feature columns at whatever the template row's original booking happened to have - since those two ratios are themselves derived from `base_fare/surge_multiplier/distance_km`, and the user just changed those three, leaving the ratios stale would feed the model an internally inconsistent row.

## Lines 183-208 -- the Customer Cancellation Risk page

```
elif page == "Customer Cancellation Risk":
    ...
    if st.button("Predict Customer Risk"):
        cancel_rate = total_cancelled / max(total_completed + total_cancelled, 1)
        row = build_template_row(df, {
            "total_completed_rides": total_completed,
            "total_cancelled_rides": total_cancelled,
            "customer_rating": customer_rating,
            "customer_cancellation_rate": cancel_rate,
            "traffic_level": traffic_level,
            "weather_condition": weather_condition,
        })
        input_df = prepare_for_prediction(row)
        pred = models["customer_cancel"].predict(input_df)[0]
        proba = models["customer_cancel"].predict_proba(input_df)[0][1]
        ...
```

`cancel_rate = total_cancelled / max(total_completed + total_cancelled, 1)` recomputes `customer_cancellation_rate` from the two ride counts the user just entered, using the same `max(..., 1)` divide-by-zero guard pattern seen throughout `feature_engineering.py` - if a user sets both counts to 0 (a brand-new customer with no history), this avoids a `ZeroDivisionError` and instead reports a 0% historical cancellation rate, which is a reasonable default for someone with no track record yet.

`predict_proba(input_df)[0][1]` pulls the model's predicted probability of class 1 ("Cancel") for the single row being predicted - `[0]` selects the first (only) row, `[1]` selects the second (positive-class) column of that row's probability pair.

## Lines 210-234 -- the Driver Delay Risk page

The fourth prediction page, added alongside the fourth model - same shape as Customer Cancellation Risk, but for driver-side inputs (`driver_rating`, `acceptance_rate`, `avg_delay_min`) plus current `traffic_level`.

`prepare_for_prediction(row, extra_drop= ["driver_delay_min"])` mirrors `train_driver_delay_model()`'s own `extra_drop` in `train_model.py` - the raw delay-minutes column has to be excluded here too, for the same leakage reason. Notice the `driver_reliability_score` override recomputes the exact weighted formula from `feature_engineering.py`'s `add_reliability_scores()` `((driver_rating * 20) * 0.4 + (acceptance_rate * 100) * 0.4 + ((1 / (1 + avg_delay_min)) * 100) * 0.2)` inline, rather than importing that formula from a shared function - a small duplication that means if the weights or formula in `feature_engineering.py` ever change, this line has to be updated by hand to match, or the dashboard's driver-delay predictions would silently start using a stale version of the score.

---

`db/schema.sql`

Not Python, but worth walking through since it's the one file defining the "data management using SQL" side of the project.

## Lines 1-13 -- header comment

Explains the overall design: four dimension tables (`customers`, `drivers`, `location_demand`, `time_features`) plus one fact table (`bookings`) referencing all four - the standard star-schema shape for this kind of transactional data, and explicitly contrasted against what the raw CSVs actually do (every booking row in the flat CSV carries its own copy of the driver's rating, acceptance rate, etc., repeated across every booking that driver ever took - exactly the redundancy a normalized schema exists to eliminate). Also notes this is written for SQLite but portable to Postgres/MySQL with minor type-name changes.

## Lines 15-50 -- the four dimension tables

```
CREATE TABLE IF NOT EXISTS customers (  
    customer_id          TEXT PRIMARY KEY,  
    customer_name        TEXT NOT NULL,  
    ...  
);
```

`IF NOT EXISTS` on every `CREATE TABLE` means running this schema file against a database that already has these tables is a safe no-op rather than an error - important since `load_db.py` calls `executescript()` with this file's contents on every run, not just the first one. `customer_id/driver_id` are declared `TEXT PRIMARY KEY` rather than an auto-incrementing integer - because the actual IDs coming from the CSVs are already meaningful strings ("`C0001`", "`D0001`"), not surrogate keys that need to be generated by the database. `location_demand`'s primary key is a *composite* key, `PRIMARY KEY (pickup_location, drop_location)` - no single column uniquely identifies a row in this table, only the pickup/drop pair together does. `time_features` uses `hour INTEGER PRIMARY KEY` - a plain integer 0-23, since that's genuinely all it takes to uniquely identify a row in this particular dimension table.

## Lines 52-73 -- the `bookings` fact table

```
CREATE TABLE IF NOT EXISTS bookings (  
    booking_id           TEXT PRIMARY KEY,  
    customer_id          TEXT NOT NULL REFERENCES customers(customer_id),  
    driver_id            TEXT NOT NULL REFERENCES drivers(driver_id),  
    ...  
    FOREIGN KEY (pickup_location, drop_location)  
        REFERENCES location_demand(pickup_location, drop_location)  
);
```

`customer_id TEXT NOT NULL REFERENCES customers(customer_id)` is SQLite's inline shorthand for declaring a foreign key at the same time as the column itself - every booking must reference a real row in `customers` (once `PRAGMA foreign_keys = ON` is set by `load_db.py`, this is actually enforced, not just documented). The composite foreign key at the bottom (`FOREIGN KEY (pickup_location, drop_location) REFERENCES location_demand(...)`) has to be declared as a separate table-level constraint rather than inline, since SQL's inline column-level shorthand only works for single-column foreign keys - a two-column reference needs the explicit `FOREIGN KEY (...) REFERENCES ...(...)` table-constraint syntax instead.

## Lines 75-82 -- indexes

```
CREATE INDEX IF NOT EXISTS idx_bookings_customer_id ON bookings(customer_id);  
CREATE INDEX IF NOT EXISTS idx_bookings_driver_id ON bookings(driver_id);  
CREATE INDEX IF NOT EXISTS idx_bookings_city ON bookings(city);  
CREATE INDEX IF NOT EXISTS idx_bookings_ride_status ON bookings(ride_status);  
CREATE INDEX IF NOT EXISTS idx_bookings_booking_time ON bookings(booking_time);
```

Five indexes, each on a column that's either a foreign key (`customer_id`, `driver_id` - looked up whenever joining back to the dimension tables) or a column the dashboard/EDA queries actually filter or group by (`city`, `ride_status`, `booking_time` - see `db/analysis_queries.sql`'s `GROUP BY city`, `WHERE ride_status = 'Cancelled'`-style filters, and the ride-volume-by-hour query's `strftime` extraction from `booking_time`). Without these, SQLite would have to scan every row in `bookings` to answer any of those queries; with them, it can jump straight to matching rows via the index instead - the difference matters much more on a real dataset with millions of rows than on this project's 6,000, but it's the correct habit to build regardless of current table size.